The VLDB Journal manuscript No.
(will be inserted by the editor)

Density Decomposition on Hypergraphs

Xiaoyu Leng · Hongchao Qin* · Rong-Hua Li · Guoren Wang

Abstract Hypergraphs capture higher-order interactions involving multiple entities. Existing decomposition models, such as k-core and nbr-k-core, rely on degree or neighborhood constraints and may overlook fine-grained density variations. We propose a (k,δ)-dense subhypergraph model based on capped hyperedge contributions. For each parameter pair, the unique dense set is the inclusionwise smallest maximizer of a penalized capped hyperedge contribution. These dense sets form a nested hierarchy characterized by egalitarian δ-orientations and hypergraph reachability. We develop three algorithms for dense subhypergraph search and two for complete density decomposition. Both decomposition algorithms use integral flow balancing: Flow-Decomp extracts density levels through strongly connected component condensation, while Reverse-Decomp uses direct reverse propagation. Once an egalitarian orientation is obtained, both recover all density levels in linear time. We further derive sandwich bounds that restrict computation to equivalent residual subproblems and develop incremental algorithms for maintaining the hierarchy under hyperedge insertions and deletions. Experiments on nine real-world hypergraphs, including datasets with over 15 million vertices, compare our methods with four established decomposition models. The results show that Reverse-Decomp improves decomposition efficiency, incremental maintenance outperforms recomputation in the tested settings, and the proposed model captures density structures overlooked by degree-based decompositions.

Keywords Hypergraphs · Density decomposition · Dense subhypergraph search · Dynamic maintenance · Network flow

*Corresponding author.

Xiaoyu Leng
Beijing Institute of Technology, Beijing, China
E-mail: lengxiaoyu@bit.edu.cn

Hongchao Qin
Beijing Institute of Technology, Beijing, China
E-mail: hcqin@bit.edu.cn

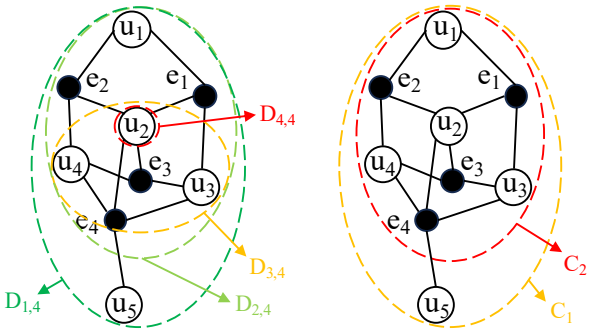
Rong-Hua Li
Beijing Institute of Technology, Beijing, China
E-mail: rhli@bit.edu.cn

Guoren Wang
Beijing Institute of Technology, Beijing, China
E-mail: wanggrbit@gmail.com

1 Introduction

Hypergraphs provide a natural abstraction for modeling higher-order relationships in which one interaction may involve more than two entities. Such structures arise in many data management and mining scenarios, including multi-party transactions, group interactions in social platforms, collaborative activities in recommendation systems, and higher-order relations in scientific domains. Compared with ordinary graphs, hypergraphs preserve the cardinality and semantics of group relations, enabling more faithful analysis of cohesive structures formed by multi-way interactions [13,41]. A fundamental task in hypergraph analysis is dense subhypergraph decomposition, which organizes vertices into nested cohesive sets for community detection, pattern discovery, anomaly detection, and structural summarization [45,37,2].

Existing hypergraph decomposition models are mainly based on local degree-style constraints. The representative k-core model requires every vertex to participate in at least k hyperedges [37], while nbr-k-core further considers the number of neighboring vertices induced by incident hyperedges [2,53]. These models are simple and scalable, but their criteria are still discrete proxies for density rather than density itself. In a hypergraph, a large hyperedge may connect many vertices while contributing weakly to local cohesion. Counting incident hyperedges or neighboring



(a) $(k,4)$-density decomposition (b) Core decomposition

Fig. 1 Density decomposition and core decomposition.

vertices cannot reliably distinguish genuinely dense sets from broad but loosely connected groups. Consequently, degree-based decompositions may produce coarse, discontinuous, or redundant layers, and may merge structurally different sets into the same level. Figure 1 illustrates this limitation. The k-core decomposition in Figure 1(b) divides the hypergraph into only two layers, where vertices in C_1 and C_2 satisfy degree requirements 1 and 2, respectively. However, this decomposition does not reveal the finer density variation inside the hypergraph. In contrast, the density-oriented decomposition in Figure 1(a) separates the same hypergraph into multiple density levels: $D_{4,4}$ corresponds to density level 4, $D_{3,4} \setminus D_{4,4}$ corresponds to density level 3, and $D_{1,4} \setminus D_{2,4}$ corresponds to density level 1. This example shows that meaningful density levels may be invisible to degree-based decomposition.

Density levels are particularly useful when they reflect meaningful group structures. In legislative co-sponsorship hypergraphs, dense sets may reveal stable coalitions; in financial transaction networks, they may indicate suspicious coordination; and in biological interaction systems, cohesive modules often arise from overlapping multi-way relations [13]. These applications call for a scalable decomposition that directly captures density and distinguishes cohesive structures across levels. However, hyperedges vary in size and may contribute to multiple vertices. Assigning their full size can overemphasize large hyperedges, while bounding their contributions requires a fair allocation among incident vertices. The challenge is to define a suitable contribution rule and efficiently construct the resulting density hierarchy.

In this paper, we propose the (k,δ)-dense subhypergraph model based on the capped hyperedge contribution function F_δ. For a vertex set X, each hyperedge contributes the portion of its budget $r_e = \min\{\delta, |e|\}$ that must be assigned within X. We define the unique dense set $D_{k,\delta}$ as the inclusionwise smallest maximizer of $F_\delta(X) - (k-1)|X|$. Supermodularity guarantees this uniqueness and nesting across parameter values, while egalitarian orientations and reachability support its computation. We develop three search algorithms, DSM-PATH, DSM-FLOW, and DSM-FLOW+, and two decomposition algorithms. Flow-Decomp computes maximum reachable loads through SCC condensation, while Reverse-Decomp uses direct reverse propagation. We further derive sandwich bounds that restrict density-level searches to equivalent residual subproblems and develop incremental algorithms for hyperedge insertions and deletions. Experiments on nine real-world hypergraphs demonstrate the efficiency and scalability of our algorithms and the advantages of incremental maintenance.

To summarize, our main contributions are as follows.

- We propose the (k,δ)-dense subhypergraph model, which enables density-oriented hierarchical decomposition of hypergraphs. In this model, k represents the integer density level of each layer, while δ bounds the contribution of each hyperedge to the density. Unlike core-based and nbr-k-core decompositions [2,53], our model directly characterizes density variations induced by multi-way relationships.
- We establish an orientation-independent supermodular characterization of (k,δ)-dense subhypergraphs, proving the uniqueness and nesting of their density levels. Using egalitarian orientations, we develop three search algorithms: DSM-PATH uses reversible hyperpaths, DSM-FLOW employs integral flow to achieve k-stability, and DSM-FLOW+ improves initialization through greedy head assignment. For complete decomposition, we design Flow-Decomp, which combines flow balancing with SCC condensation, and Reverse-Decomp, which replaces SCC computation with direct reverse propagation. Both recover all density levels in $O(n+m+L)$ additional time after global balancing. We further establish a sandwich theorem that bounds each dense set before balancing and reduces its computation to an equivalent residual subproblem.
- We evaluate the algorithms on nine real-world hypergraphs, including datasets with over 15 million vertices, considering search efficiency, decomposition time and memory, hierarchy granularity, scalability, and dynamic maintenance. We compare our methods with four established decomposition models on five datasets. Reverse-Decomp consistently outperforms Flow-Decomp on these datasets, achieving speedups of $3.11\times$ and $2.39\times$ on SB and HB, respectively.
- The code is available at `https://github.com/xiaoyu-ll/IDH`.

2 Problem Definition

Let $\mathscr{H} = (V,E)$ be an undirected and unweighted hypergraph, where V is the vertex set and E is an indexed family of nonempty hyperedges. Each hyperedge $e \in E$ is a

set $e \subseteq V$, $|e| \geq 1$. Identical hyperedges may occur as distinct indexed records, but a vertex cannot appear more than once within a single hyperedge. Let $n = |V|$ and $m = |E|$ denote the numbers of vertices and hyperedges, respectively. We use $d^E_{\max}$ and $d^E_{\min}$ to denote the maximum and minimum hyperedge sizes in $\mathscr{H}$, and define the average hyperedge size as $\bar{d}_e = \frac{1}{m}\sum_{e\in E}|e|$ when $m > 0$. For a vertex $u \in V$, its degree d_u is the number of hyperedges containing u. For a vertex set $S \subseteq V$, $E[S]$ denotes the set of hyperedges fully contained in S. Let $\delta \geq 1$ be an integer and write $r_e = \min\{\delta, |e|\}$. When $E \neq \emptyset$, it suffices for a complete decomposition to examine $1 \leq \delta \leq d^E_{\max}$, since larger values have identical head capacities. If $E = \emptyset$, every vertex has IDN zero. We write $L = \sum_{e\in E}|e|$ for the input incidence count and $R_\delta = \sum_e r_e \leq m\delta$ for total allocated heads.

2.1 Capped Hyperedge Contribution

Definition 1 (Capped Hyperedge Contribution) For $X \subseteq V$ and an integer parameter $\delta \geq 1$, with $r_e = \min\{\delta, |e|\}$, define $F_\delta(X) = \sum_{e\in E}\max\{0, r_e - |e \setminus X|\}$.

Each hyperedge e contributes r_e units, allocated to distinct incident vertices. Since at most $|e \setminus X|$ units can be assigned outside X, its minimum contribution to X is $\max\{0, r_e - |e \setminus X|\}$. This bound is tight, as the contribution can be minimized by assigning as many units as possible outside X. Thus, $F_\delta(X)$ captures the total guaranteed contribution within X, independently of any particular orientation of the hypergraph. Identical hyperedges are treated as distinct occurrences and contribute separately. In particular, $F_\delta(\emptyset) = 0$ and $F_\delta(V) = R_\delta$.

Definition 2 (Capped Density) For every nonempty $X \subseteq V$, define $\rho_\delta(X) = \frac{F_\delta(X)}{|X|}$.

Unlike degree-density, which considers only fully contained hyperedges, this density also accounts for the capped contributions of partially contained hyperedges that must be assigned within the vertex set.

The function F_δ is monotone and supermodular. For any $A, B \subseteq V$, we have $F_\delta(A) + F_\delta(B) \leq F_\delta(A\cup B) + F_\delta(A\cap B)$. To see this, the contribution of each hyperedge e can be expressed as $\max\{0, |e\cap X| - (|e| - r_e)\}$, which is nondecreasing and convex in $|e\cap X|$. Consequently, adding a vertex to a larger set yields a contribution gain no smaller than adding it to a subset. Since these properties hold for each hyperedge, they are preserved under summation, establishing the monotonicity and supermodularity of F_δ.

2.2 Dense Sets and Hypergraph Decomposition

Definition 3 ((k,δ)-Dense Subhypergraph) For integers $k \geq 0$ and $\delta \geq 1$, let $Q_{k,\delta}(X) = F_\delta(X) - (k-1)|X|$, $\quad X \subseteq V$. The dense vertex set is $D_{k,\delta} = \bigcap \arg\max_{X\subseteq V} Q_{k,\delta}(X)$. The (k,δ)-dense subhypergraph is $\mathscr{H}[D_{k,\delta}]$. We also write $D_{k,\delta}$ for its vertex set when no ambiguity arises.

Theorem 1 (Uniqueness of Dense Set) *For every $k \geq 0$ and $\delta \geq 1$, the intersection in Definition 3 is a maximizer of $Q_{k,\delta}$ and is its unique inclusionwise smallest maximizer. Hence, $D_{k,\delta}$ is uniquely defined.*

Proof. A maximizer exists because V is finite. Subtracting the modular function $(k-1)|X|$ preserves supermodularity. If A and B both attain the maximum q, then $2q \leq Q_{k,\delta}(A\cup B) + Q_{k,\delta}(A\cap B) \leq 2q$. Both their union and intersection are maximizers. Repeated intersection over the finite maximizing family gives a maximizer contained in every other one, proving the claim. □

The uniqueness criterion in Theorem 1 resolves ties at integer density levels by selecting the inclusionwise smallest maximizer. In particular, $D_{0,\delta} = V$, whereas an isolated vertex belongs to no $D_{k,\delta}$ with $k \geq 1$. For $k = 1$, adding an isolate leaves the objective unchanged, and the intersection excludes it. If $K = \max_v d_v$, with $K = 0$ for an empty vertex set, then $F_\delta(X) \leq K|X|$ and $D_{K+1,\delta} = \emptyset$.

Theorem 2 (Nested dense sets) *For any $\delta \geq 1$ and integers $k^+ \geq k \geq 0$, $D_{k^+,\delta} \subseteq D_{k,\delta}$.*

Proof. Equality of the two level parameters is immediate. Otherwise let $A = D_{k,\delta}$, $B = D_{k^+,\delta}$ and $Z = B \setminus A$. Optimality against $A\cup B$ and $A\cap B$ gives

$$F_\delta(A\cup B) - F_\delta(A) \leq (k-1)|Z|,$$
$$F_\delta(B) - F_\delta(A\cap B) \geq (k^+ - 1)|Z|.$$

Supermodularity bounds the second left side by the first. Since $k^+ > k$, these inequalities force $Z = \emptyset$. □

Definition 4 (Density Decomposition and IDN) For a given δ, define the indexed layers and integral dense numbers by $L_{k,\delta} = D_{k,\delta} \setminus D_{k+1,\delta}$, $\bar{r}^\delta_v = \max\{k \geq 0 : v \in D_{k,\delta}\}$. The density decomposition at δ is the collection of nonempty indexed layers $(k, L_{k,\delta})$, equivalently the complete IDN array. Empty integer levels retain their indices. Computing the full parameter family repeats this definition for $1 \leq \delta \leq d^E_{\max}$ when $E \neq \emptyset$.

Nesting, $D_{0,\delta} = V$ and $D_{K+1,\delta} = \emptyset$ make the nonempty layers a partition of V. Isolates have IDN zero. If $E = \emptyset$, every IDN is zero for every positive δ.

2.3 Density Properties

Theorem 3 (Removal and Addition Bounds) *A set $D \subseteq V$ equals $D_{k,\delta}$ if and only if, for every nonempty $X \subseteq D$ and every $Y \subseteq V \setminus D$,*

$$F_\delta(D) - F_\delta(D\setminus X) > (k-1)|X|,$$
$$F_\delta(D\cup Y) - F_\delta(D) \leq (k-1)|Y|.$$

Proof. We first prove necessity. Since $D_{k,\delta}$ is the unique inclusionwise smallest maximizer of $Q_{k,\delta}$, every proper subset $D \setminus X$ has a strictly smaller objective value. Thus, $Q_{k,\delta}(D) > Q_{k,\delta}(D \setminus X)$, which gives the strict removal inequality. Similarly, the optimality of D implies $Q_{k,\delta}(D) \geq Q_{k,\delta}(D \cup Y)$, yielding the addition inequality. For sufficiency, suppose D satisfies both inequalities. Let $Z \subseteq V$ be arbitrary and write $Q = Q_{k,\delta}$. By supermodularity, $Q(D)+Q(Z) \leq Q(D \cup Z)+Q(D \cap Z)$. The addition inequality gives $Q(D \cup Z) \leq Q(D)$, while the removal inequality implies $Q(D \cap Z) \leq Q(D)$, with strict inequality whenever $D \setminus Z \neq \emptyset$. Consequently, $Q(Z) \leq Q(D \cap Z) \leq Q(D)$. Hence, D is a maximizer of Q. Moreover, equality requires $D \subseteq Z$, so every maximizer contains D. Therefore, D is the unique inclusionwise smallest maximizer, and $D = D_{k,\delta}$. □

Definition 5 (Layer Density) For a nonempty layer $L_{k,\delta}$, define

$$\lambda_{k,\delta} = \frac{F_\delta(D_{k,\delta}) - F_\delta(D_{k+1,\delta})}{|L_{k,\delta}|}.$$

This density measures the contribution of a layer relative to its higher-level set. It generally differs from the stand-alone density $\rho_\delta(L_{k,\delta}) = F_\delta(L_{k,\delta})/|L_{k,\delta}|$, since F_δ is not additive. All active hyperedges are considered, including partially contained ones. Empty layers have undefined density, while the zero layer has density zero.

Lemma 1 *For nonempty $X \subseteq D_{k,\delta}$ and $Y \subseteq V \setminus D_{k,\delta}$, the removal and addition densities satisfy*

$$\frac{F_\delta(D) - F_\delta(D \setminus X)}{|X|} > k-1,$$
$$\frac{F_\delta(D \cup Y) - F_\delta(D)}{|Y|} \leq k-1,$$

where $D = D_{k,\delta}$. These are density inequalities, not inequalities for the stand-alone density of an arbitrary subset.

Proof. Divide the two capped hyperedge contribution inequalities by the positive set sizes. □

Lemma 2 *Every nonempty layer satisfies $k-1 < \lambda_{k,\delta} \leq k$.*

Proof. Apply the strict removal inequality to $D_{k,\delta}$ with $X = L_{k,\delta}$, then the addition inequality to $D_{k+1,\delta}$ with $Y = L_{k,\delta}$. Divide by $|L_{k,\delta}|$. □

Proposition 1 (Direct layer evaluation) *For each edge occurrence, order its endpoint IDNs as $a_{e,1} \leq \cdots \leq a_{e,|e|}$. Then*

$$F_\delta(D_{k,\delta}) = \sum_{e \in E} \sum_{i=1}^{r_e} \mathbf{1}\{a_{e,i} \geq k\},$$
$$|L_{k,\delta}|\lambda_{k,\delta} = \sum_{e \in E} \sum_{i=1}^{r_e} \mathbf{1}\{a_{e,i} = k\}.$$

Consequently $\sum_{k:L_{k,\delta} \neq \emptyset} |L_{k,\delta}|\lambda_{k,\delta} = R_\delta$.

Proof. The number of endpoints of e outside $D_{k,\delta}$ is $|\{i : a_{e,i} < k\}|$. Its contribution is therefore the number of the first r_e ordered values that are at least k. Subtract the formula at $k+1$ to obtain the layer numerator. The sum of the numerators equals the total edge budget. □

This formula computes the capped contribution density of each layer directly from the IDNs, without reconstructing an orientation. It accounts for partially contained hyperedges and repeated edge occurrences.

2.4 Directed Hypergraphs and Orientations

To represent the allocation of density contributions among vertices, we introduce hyperedge orientations. An orientation of $\mathscr{H}$ transforms each undirected hyperedge $e \in E$ into a directed hyperedge $\vec{e} = (T_e, H_e)$, where T_e and H_e are the source and target sets, respectively, satisfying $T_e \cap H_e = \emptyset$ and $T_e \cup H_e = e$. The resulting directed hypergraph is denoted by $\vec{\mathscr{H}} = (V, \vec{E})$. Figure 2(b)–(c) illustrates two orientations of the hypergraph in Figure 2(a). In Figure 2(b), $\vec{e}_4 = (\{u_3, u_4, u_5\}, \{u_2\})$ has source vertices u_3, u_4, u_5 and target vertex u_2. Given $\vec{\mathscr{H}}$, the indegree of a vertex u is defined as $\vec{d}_u(\vec{\mathscr{H}}) = \left|\{\vec{e} = (T_e, H_e) \in \vec{E} \mid u \in H_e\}\right|$. We simply write $\vec{d}_u$ when the orientation is clear.

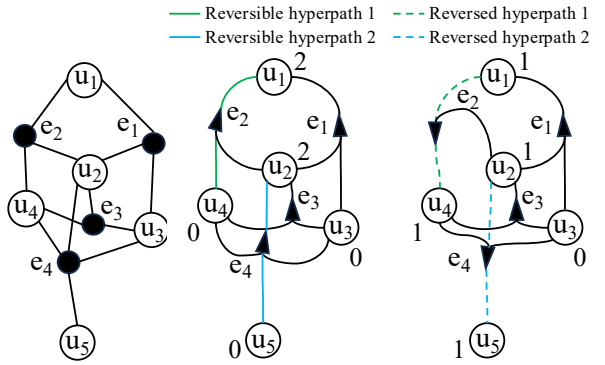
Definition 6 (δ-Orientation) Given a hypergraph $\mathscr{H} = (V, E)$ and its orientation $\vec{\mathscr{H}} = (V, \vec{E})$, $\vec{\mathscr{H}}$ is called a δ-orientation of $\mathscr{H}$ if for every directed hyperedge $\vec{e} = (T_e, H_e) \in \vec{E}$ corresponding to an undirected hyperedge $e \in E$, we have $|H_e| = \min\{\delta, |e|\}$, where δ is a positive integer. The allowed head set is all of e when $|e| \leq \delta$.

2.5 Hyperpaths and Egalitarian Orientation

We next define hyperpaths and reversible hyperpaths, which formalize how density contributions can be transferred and balanced under a δ-orientation.

Definition 7 (Hyperpath, Reversible Hyperpath) An *incidence-simple directed hyperpath* from s to t is an alternating path $s = u_0, \vec{e}_1, u_1, \ldots, \vec{e}_\ell, u_\ell = t$ in which each $\vec{e}_i = (T_{e_i}, H_{e_i})$ is distinct, $u_{i-1} \in T_{e_i}$, and $u_i \in H_{e_i}$. The path is *reversible* if $\vec{d}_t - \vec{d}_s \geq 2$.

We write $s \rightsquigarrow t$ when s can reach t. *Reorienting* an incidence-simple path means swapping u_{i-1} and u_i between the tail and head sets of every traversed hyperedge, i.e., $H_{e_i} \leftarrow (H_{e_i} \setminus \{u_i\}) \cup \{u_{i-1}\}$. It does not mean reversing all incidences of a hyperedge. Each $|H_{e_i}| = r_{e_i}$ is preserved; the indegree of s increases by one, that of t decreases by one, and every intermediate vertex has unchanged indegree. Consequently the sum of squared indegrees strictly decreases on a reversible path. An egalitarian orientation admits no such path.



(a) initial hypergraph (b) 1-orientation (c) egalitarian 1-orientation

Fig. 2 Arbitrary and egalitarian 1-orientations.

Definition 8 (Egalitarian δ-Orientation) A δ-orientation $\vec{\mathscr{H}}$ is said to be an *egalitarian δ-orientation* if there exists no reversible hyperpath in $\vec{\mathscr{H}}$.

Lemma 3 *For any hypergraph $\mathscr{H} = (V,E)$, any integer parameter $\delta \geq 1$, there exists an egalitarian δ-orientation $\vec{\mathscr{H}}$.*

Proof. A valid δ-orientation can be constructed by selecting $r_e = \min\{\delta, |e|\}$ target vertices for each hyperedge $e \in E$ and assigning the remaining vertices to its source set. Starting from this orientation, we repeatedly reverse any reversible hyperpath. By Definition 7, reversing a path from s to t increases $\vec{d}_s$ by one and decreases $\vec{d}_t$ by one, while preserving the target-set size of every hyperedge. Hence, each reversal maintains a valid δ-orientation.

To prove termination, consider the potential function $\Psi(\vec{\mathscr{H}}) = \sum_{u\in V} \vec{d}_u^2$. For a reversible hyperpath from s to t, let $a = \vec{d}_s$ and $b = \vec{d}_t$, where $b - a \geq 2$. After reversal, the change in potential is

$$\Delta\Psi = (a+1)^2 + (b-1)^2 - a^2 - b^2 = -2(b-a-1) < 0.$$

Thus, every reversal strictly decreases the nonnegative integer potential Ψ, and the process must terminate. At termination, no reversible hyperpath remains, yielding an egalitarian δ-orientation by Definition 8. □

Example 1 Figure 2(b) illustrates an arbitrary 1-orientation containing two reversible hyperpaths: $u_4 \rightsquigarrow u_1$ through $(u_4, \vec{e}_2, u_1)$ and $u_5 \rightsquigarrow u_2$ through $(u_5, \vec{e}_4, u_2)$. Since $\vec{d}_{u_1} - \vec{d}_{u_4} = 2$ and $\vec{d}_{u_2} - \vec{d}_{u_5} = 2$, both paths satisfy the reversal condition. Reversing each path transfers one unit of indegree from its terminal vertex to its starting vertex, reducing the corresponding indegree gap by two while preserving the target-set size of each hyperedge. The resulting orientation, shown in Figure 2(c), contains no reversible hyperpath and is therefore an egalitarian 1-orientation.

2.6 Orientation Bounds and Reachability

Orientations are a computational representation of the preceding set-function definitions. They are not needed to define F_δ, $D_{k,\delta}$, the layers, or the IDNs.

For any valid δ-orientation, each edge assigns at least $\max\{0, r_e - |e \setminus X|\}$ heads to X. Assigning heads outside X first attains this lower bound independently on every edge. Therefore $F_\delta(X) = \min_{\text{valid } \delta\text{-orientations}} \sum_{v\in X} \vec{d}_v$. For a particular orientation, $F_\delta(X) \leq \sum_{v\in X} \vec{d}_v$. Equality holds if and only if no edge has a tail outside X and a head inside X, equivalently when X is predecessor-closed. If both such incidences exist, exchanging their roles strictly reduces the assigned load in X; otherwise every edge already attains its minimum.

Given an egalitarian δ-orientation, let $S_k = \{v : \vec{d}_v \geq k\}$. We define the set of vertices that can reach S_k as

$$C_{k,\delta} = \{v \in V : v \rightsquigarrow s \text{ for some } s \in S_k\}.$$

Reachability includes zero-length paths, so $S_k \subseteq C_{k,\delta}$.

Lemma 4 (Reachability-set bounds) *No edge has a tail outside $C_{k,\delta}$ and a head inside it. Every vertex in $C_{k,\delta}$ has indegree at least $k-1$, and every vertex outside it has indegree at most $k-1$.*

Proof. Suppose a hyperedge has a tail vertex $u \notin C_{k,\delta}$ and a head vertex $v \in C_{k,\delta}$. Since $u \rightsquigarrow v$ and v can reach some seed $s \in S_k$, we have $u \rightsquigarrow s$, contradicting $u \notin C_{k,\delta}$. Next, if a vertex $v \in C_{k,\delta}$ has $\vec{d}_v \leq k-2$, it can reach a seed $s \in S_k$ with $\vec{d}_s \geq k$. Hence, $\vec{d}_s - \vec{d}_v \geq 2$, implying the existence of a reversible hyperpath and contradicting egalitarianity. Finally, every vertex $v \notin C_{k,\delta}$ must satisfy $\vec{d}_v \leq k-1$, since otherwise v would itself be a seed in S_k and thus belong to $C_{k,\delta}$. □

Lemma 5 (Dense-Set Recovery) *If a valid δ-orientation has no path $s \rightsquigarrow t$ with $\vec{d}_s \leq k-2$ and $\vec{d}_t \geq k$, then the set of vertices that can reach $\{v : \vec{d}_v \geq k\}$ equals $D_{k,\delta}$.*

Proof. Let $S = \{v : \vec{d}_v \geq k\}$ and $C = \{v : v \rightsquigarrow s \text{ for some } s \in S\}$. By the assumed path condition, every vertex in C has indegree at least $k-1$, while every vertex outside C has indegree at most $k-1$. Moreover, C is predecessor-closed, implying $F_\delta(C) = \sum_{v\in C} \vec{d}_v$. For any nonempty $X \subseteq C$, let $B = C \setminus X$. We have $F_\delta(C) - F_\delta(B) = \sum_{v\in X} \vec{d}_v + \left(\sum_{v\in B} \vec{d}_v - F_\delta(B)\right)$. If X contains a vertex in S, the first term exceeds $(k-1)|X|$, while the second is nonnegative. Otherwise, every vertex in X has indegree $k-1$ and can reach a seed in B. Hence, some path from X to B crosses a hyperedge with a tail outside B and a head inside B, implying $F_\delta(B) < \sum_{v\in B} \vec{d}_v$. Thus, the strict removal inequality holds in both cases. For any $Y \subseteq V \setminus C$, the allocation bound and the indegrees outside C give $F_\delta(C \cup Y) - F_\delta(C) \leq \sum_{v\in Y} \vec{d}_v \leq (k-1)|Y|$. Therefore, C satisfies both the removal and addition bounds. By Theorem 3, we conclude that $C = D_{k,\delta}$. □

Theorem 4 (IDNs via Reachability) *For any egalitarian δ-orientation, the IDN of each vertex v is given by $\bar{r}_v^\delta = \max_{w:\, v \rightsquigarrow w} \vec{d}_w$, where reachability includes v itself.*

Proof. Consider an egalitarian δ-orientation. Let $R(v)$ denote the set of vertices reachable from v, including v itself. By the equivalence $D_{k,\delta} = C_{k,\delta}$, a vertex v belongs to $D_{k,\delta}$ precisely when there exists some $w \in R(v)$ with $\vec{d}_w \geq k$. Let $M(v) = \max_{w \in R(v)} \vec{d}_w$ denote the maximum indegree among vertices reachable from v. It follows that $v \in D_{k,\delta}$ holds if and only if $k \leq M(v)$. Since the IDN of v is the largest density level containing v, we obtain $\bar{r}_v^\delta = \max\{k \geq 0 : v \in D_{k,\delta}\} = \max_{w \in R(v)} \vec{d}_w$. Therefore, the IDN of each vertex equals its maximum reachable indegree. □

Remark on δ. The parameter δ determines the contribution of each hyperedge to its incident vertices. No hyperedge is removed: if $|e| > \delta$, then δ incident vertices receive one unit of contribution; otherwise, all incident vertices do. Thus, a larger δ allows each hyperedge to support more vertices, whereas a smaller δ imposes a stricter contribution cap. The number of distinct occupied IDN layers need not increase monotonically with δ.

Objective and Relation to Cores. Our density hierarchy is defined by maximizing the penalized capped hyperedge contribution $Q_{k,\delta}$, rather than the conventional induced degree-density $\rho(X)$. The preceding orientation-based characterization provides an equivalent way to compute these dense sets. When $\delta = 1$, every vertex in $D_{k,1}$ is incident to at least k hyperedges fully contained in $D_{k,1}$. Thus, $D_{k,1}$ is contained in the hypergraph k-core. For ordinary loopless graphs, a stronger relationship holds: $C_{2k} \subseteq D_{k,1} \subseteq C_k$, where C_k denotes the conventional k-core. To establish $D_{k,1} \subseteq C_k$ for $k \geq 1$, recall that $D_{k,1}$ is predecessor-closed. Hence, every incoming edge of a vertex in $D_{k,1}$ is fully contained in this set. Each seed has indegree at least k and therefore at least k incident edges. Each nonseed has indegree $k-1$ and can reach a seed through a directed path. The first edge on this path is an additional incident edge contained in $D_{k,1}$. Consequently, every vertex in $D_{k,1}$ has induced degree at least k, proving the inclusion. For the reverse inclusion $C_{2k} \subseteq D_{k,1}$, let $A = C_{2k} \setminus D_{k,1}$. Since $D_{k,1}$ is predecessor-closed, every edge between A and $C_{2k} \cap D_{k,1}$ must be oriented toward A. Moreover, each edge within A contributes one unit of indegree to A. Therefore, the total indegree allocated to A is at least half its degree sum within C_{2k}. As every vertex in C_{2k} has induced degree at least $2k$, the total allocated indegree in A is at least $k|A|$. However, every vertex outside $D_{k,1}$ has indegree at most $k-1$, implying a total indegree of at most $(k-1)|A|$. This contradiction forces $A = \emptyset$. For $k = 0$, both inclusions follow directly from $D_{0,1} = C_0 = V$.

2.7 Density and Boundary Guarantee

We relate the capped hyperedge contribution to fully-contained-edge degree-density.

Definition 9 (Degree-Density) For nonempty $X \subseteq V$, let $d_v(X) = |\{e \in E[X] : v \in e\}|$ and define

$$\rho(X) = \frac{1}{|X|} \sum_{v \in X} d_v(X) = \frac{\sum_{e \in E[X]} |e|}{|X|}.$$

Definition 10 (Capped Hyperedge Contribution Internalization Coefficient) For a nonempty $X \subseteq V$ with $F_\delta(X) > 0$, define $\theta(X) = \frac{\sum_{e \in E[X]} r_e}{F_\delta(X)}$. Set $\theta(X) = 0$ when $F_\delta(X) = 0$. The coefficient depends only on the hypergraph, X and δ, and lies in $[0,1]$.

Every hyperedge fully contained in X contributes its entire budget r_e to $F_\delta(X)$, while partially contained hyperedges may contribute additional nonnegative amounts. Therefore, the total contribution from internal hyperedges cannot exceed $F_\delta(X)$, yielding $0 \leq \sum_{e \in E[X]} r_e \leq F_\delta(X)$. It follows that $0 \leq \theta(X) \leq 1$ whenever $F_\delta(X) > 0$. The same bound holds when $F_\delta(X) = 0$ by definition.

Lemma 6 *For every nonempty $D = D_{k,\delta}$,*

$$\rho_\delta(D) \geq (k-1) + \frac{1}{|D|}.$$

Proof. Let $D = D_{k,\delta}$ be a nonempty dense set. Applying the strict removal inequality with $X = D$, we obtain $F_\delta(D) - F_\delta(\emptyset) > (k-1)|D|$. Since $F_\delta(\emptyset) = 0$, this reduces to $F_\delta(D) > (k-1)|D|$. Moreover, both $F_\delta(D)$ and $(k-1)|D|$ are integers, so the strict inequality implies $F_\delta(D) \geq (k-1)|D| + 1$. Dividing both sides by $|D| > 0$ yields $\rho_\delta(D) = \frac{F_\delta(D)}{|D|} \geq k - 1 + \frac{1}{|D|}$, which establishes the claimed lower bound. □

Lemma 7 *For every nonempty $X \subseteq V$, $\rho(X) \geq \theta(X)\rho_\delta(X)$.*

Proof. Let $X \subseteq V$ be nonempty. If $F_\delta(X) = 0$, then $\theta(X) = 0$ by definition, and the inequality holds since $\rho(X) \geq 0$. Otherwise, recall that $r_e = \min\{\delta, |e|\} \leq |e|$ for every hyperedge e. Summing over all hyperedges fully contained in X, we obtain $\sum_{e \in E[X]} |e| \geq \sum_{e \in E[X]} r_e = \theta(X) F_\delta(X)$. Dividing both sides by $|X|$ and applying the definitions of degree-density and capped density gives $\rho(X) = \frac{\sum_{e \in E[X]} |e|}{|X|} \geq \theta(X) \frac{F_\delta(X)}{|X|} = \theta(X)\rho_\delta(X)$. This establishes the desired inequality. □

Theorem 5 (Capped Hyperedge Density Guarantee) *For every nonempty $D = D_{k,\delta}$,*

$$\rho(D) \geq \theta(D)\rho_\delta(D) \geq \theta(D)\left(k - 1 + \frac{1}{|D|}\right).$$

Proof. Combine Lemmas 6 and 7, using $\theta(D) \geq 0$. □

Definition 11 (Boundary-to-Volume Ratio) For $X \subseteq V$ with $\mathrm{vol}(X) = \sum_{v\in X} d_v > 0$, let $b(X)$ be the number of hyperedges meeting both X and $V \setminus X$, and define $\phi_{\mathrm{out}}(X) = b(X)/\mathrm{vol}(X)$. This is a one-sided boundary ratio, not the standard symmetric conductance.

Theorem 6 (Boundary Ratio Bound) *For every nonempty X with $\mathrm{vol}(X) > 0$, writing $\bar{d}(X) = \mathrm{vol}(X)/|X|$,*

$$\phi_{\mathrm{out}}(X) \le 1 - \frac{\rho(X)}{\bar{d}(X)}.$$

In particular, if $D_{k,\delta} \neq \emptyset$ and $\mathrm{vol}(D_{k,\delta}) > 0$, then

$$\phi_{\mathrm{out}}(D_{k,\delta}) \le 1 - \frac{\theta(D_{k,\delta})\rho_\delta(D_{k,\delta})}{\bar{d}(D_{k,\delta})}.$$

Proof. Consider any nonempty vertex set $X \subseteq V$ with $\mathrm{vol}(X) > 0$. The incidences within X can be divided into those belonging to fully contained hyperedges and those belonging to crossing hyperedges. The former contribute $\sum_{e\in E[X]} |e| = \rho(X)|X|$, while each crossing hyperedge contributes at least one incidence to X. Therefore, $b(X) \le \mathrm{vol}(X) - \rho(X)|X|$. Dividing both sides by $\mathrm{vol}(X) = \bar{d}(X)|X|$, we obtain $\phi_{\mathrm{out}}(X) = \frac{b(X)}{\mathrm{vol}(X)} \le 1 - \frac{\rho(X)}{\bar{d}(X)}$. For a nonempty dense set $D_{k,\delta}$ with positive volume, Theorem 5 gives $\rho(D_{k,\delta}) \ge \theta(D_{k,\delta})\rho_\delta(D_{k,\delta})$. Substituting this bound into the inequality above yields the result. □

The symmetric ratio $b(X)/\min\{\mathrm{vol}(X), \mathrm{vol}(V \setminus X)\}$ is not bounded this way unless the denominator is $\mathrm{vol}(X)$ (or the corresponding volume-side condition is imposed).

2.8 Search and Decomposition

Given a hypergraph, a nonnegative density level k and a positive integer δ, the search task returns $D_{k,\delta}$. The decomposition task takes δ as input and computes the canonical IDN array, hence every nested set $D_{k,\delta}$. Computing all effective parameter values repeats the procedure for $1 \le \delta \le d^E_{\max}$. The challenge is to balance contributions without enumerating orientations and recover the entire hierarchy without repeating a search for every k.

3 Dense Subhypergraph Search

Let $L = \sum_{e\in E} |e|$ and $R_\delta = \sum_{e\in E} \min\{\delta, |e|\} \le m\delta$. The directed incidence representation contains $O(n+m+L)$ vertices and arcs. Every exchange below is performed on an incidence-simple hyperpath by swapping one tail and one head of each traversed hyperedge; exchanging entire hyperedge head and tail sets would not preserve the δ-orientation.

Algorithm 1: DSM-PATH$(\mathscr{H}, k, \delta)$

Data: $\mathscr{H} = (V, E)$, integers $k \ge 0$, $\delta \ge 1$
Result: $D_{k,\delta}$

```
Construct any valid δ-orientation ℋ⃗;
while some incidence-simple reversible hyperpath s ⇝ t exists do
    Swap one tail/head pair along every traversed hyperedge;
S ← {v : d⃗_v ≥ k};
return {v : v ⇝ s for some s ∈ S};
```

3.1 The Path-Based Algorithm: DSM-PATH

DSM-PATH constructs an egalitarian δ-orientation by repeatedly identifying and reversing reversible hyperpaths. Starting from an arbitrary δ-orientation, we first construct a directed incidence graph by introducing a virtual vertex z_e for each hyperedge e. For every $u \in T_e$ and $v \in H_e$, we add directed arcs $u \to z_e$ and $z_e \to v$, respectively. This representation preserves reachability in the directed hypergraph. To find a reversible hyperpath, we compute the strongly connected components (SCCs) of the incidence graph and contract each component into a single vertex, obtaining a directed acyclic graph (DAG). Each component is assigned the maximum indegree of its original vertices. By propagating these values in reverse topological order, we obtain the maximum reachable indegree $M(v)$ for every vertex v. If some vertex s satisfies $M(s) \ge \vec{d}_s + 2$, a reversible hyperpath exists. We then use breadth-first search (BFS) to find a path from s to a reachable vertex t with $\vec{d}_t \ge \vec{d}_s + 2$.

Reversing this path transfers one unit of indegree from t to s while preserving the target-set size of every traversed hyperedge. The algorithm repeats this process until no reversible hyperpath remains, yielding an egalitarian δ-orientation. Each round of SCC computation, reachability propagation, and path reconstruction takes $O(n+m+L)$ time, where $L = \sum_{e\in E} |e|$.

Theorem 7 (Correctness and a conservative PATH bound) *Algorithm 1 returns $D_{k,\delta}$. With SCC-based witness detection, its time and space bounds are $O((R^2_\delta + 1)(n+m+L))$ and $O(n+m+L)$.*

Proof. Each path reversal preserves the target-set size of every hyperedge and changes only the indegrees of its endpoints. For a reversible hyperpath from s to t, the indegree of s increases by one while that of t decreases by one, preserving a valid δ-orientation. Consider the potential function $\Psi = \sum_{v\in V} \vec{d}^2_v$. Since $\vec{d}_t - \vec{d}_s \ge 2$, each reversal decreases the potential by $-\Delta\Psi = 2(\vec{d}_t - \vec{d}_s - 1) \ge 2$. As $0 \le \Psi \le R^2_\delta$, at most $O(R^2_\delta)$ reversals are performed. Each iteration uses SCC-based reachability analysis and BFS to identify and reconstruct a reversible hyperpath, taking $O(n+m+L)$ time.

Thus, the overall time complexity is $O((R_\delta^2+1)(n+m+L))$, with $O(n+m+L)$ space. Upon termination, no reversible hyperpath remains, yielding an egalitarian δ-orientation. By Lemma 5, the vertices that can reach a seed of indegree at least k form $D_{k,\delta}$, proving correctness. □

3.2 The Flow-Based Algorithm: DSM-FLOW

DSM-PATH computes a globally egalitarian orientation, but finding $D_{k,\delta}$ requires only eliminating directed hyperpaths from vertices with indegree at most $k-2$ to those with indegree at least k. Reversing such a path increases the indegree of its starting vertex by one and decreases that of its final vertex by one. Instead of processing these paths individually, DSM-FLOW uses maximum flow to perform multiple reorientations in a single computation.

Reorientation flow network. Given a valid δ-orientation, we construct a directed flow network $\mathcal{N}=(V_N,A,c)$ with a source s and a sink t. Each hyperedge e is represented by a factor vertex z_e. A directed two-arc segment $u \to z_e \to v$, with $u \in T_e$ and $v \in H_e$, represents a possible exchange of a head from v to u. We use $q=k-1$ as the pivot separating vertices that can receive additional contributions from vertices that can release them.

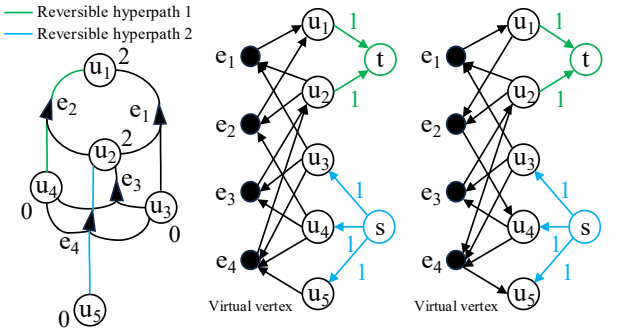
Definition 12 (Reorientation Flow Network) For a valid δ-orientation and $q=k-1$, let $V_N = V \cup \{z_e : e \in E\} \cup \{s,t\}$. The arcs and their capacities are defined as follows:

- For each $e \in E$, add $u \to z_e$ for every $u \in T_e$ and $z_e \to v$ for every $v \in H_e$, each with unit capacity.
- For each u with $\vec{d}_u < q$, add $s \to u$ with capacity $q - \vec{d}_u$.
- For each v with $\vec{d}_v > q$, add $v \to t$ with capacity $\vec{d}_v - q$.

The network has $n+m+2$ vertices and $O(n+L)$ arcs, where $L = \sum_{e \in E} |e|$.

The source and sink capacities bound the number of contributions that low- and high-indegree vertices may exchange. In particular, every s–t augmenting path corresponds to a directed hyperpath from a vertex below q to a vertex above q. Sending one unit of flow along this path transfers one head contribution in the opposite direction, while preserving the head budget of each traversed hyperedge. Because all hyperedge-incidence arcs have unit capacity, an integral maximum flow can be converted back to a valid δ-orientation.

Example 2 Consider the 1-orientation in Figure 3(a), where the indegrees of $u_1,\dots,u_5$ are $(2,2,0,0,0)$. For $k=2$, the pivot is $q=1$. In Figure 3(b), the source s is connected to u_3,u_4,u_5, each with capacity one, and u_1,u_2 are connected to the sink t, also with unit capacity. For each hyperedge e_i, the factor vertex z_{e_i} connects its tail vertices to its head vertices through unit-capacity arcs. For example, the network



(a) 1-orientation (b) Reorientation flow network (c) Orientation after flow

Fig. 3 Reorientation of a hypergraph using maximum flow for $k=2$ and $\delta=1$.

Algorithm 2: DSM-FLOW$(\mathscr{H},k,\delta)$

Data: $\mathscr{H}=(V,E)$, integers $k \geq 0$, $\delta \geq 1$
Result: $D_{k,\delta}$
if $k=0$ **then**
  return V;
Construct any valid δ-orientation $\vec{\mathscr{H}}$;
$q \leftarrow k-1$, $V_N \leftarrow V \cup \{z_e : e \in E\} \cup \{s,t\}$, $A \leftarrow \emptyset$;
foreach $e \in E$ **do**
  foreach $u \in T_e$ **do**
    Add $u \to z_e$ to A with capacity 1;
  foreach $v \in H_e$ **do**
    Add $z_e \to v$ to A with capacity 1;
foreach $v \in V$ **do**
  if $\vec{d}_v < q$ **then**
    Add $s \to v$ to A with capacity $q - \vec{d}_v$;
  if $\vec{d}_v > q$ **then**
    Add $v \to t$ to A with capacity $\vec{d}_v - q$;
$f \leftarrow \text{MaxFlow}(V_N,A,c,s,t)$;
foreach $e \in E$ **do**
  $U_e \leftarrow \{u \in T_e : f(u,z_e)=1\}$;
  $W_e \leftarrow \{v \in H_e : f(z_e,v)=1\}$;
  $H_e \leftarrow (H_e \setminus W_e) \cup U_e$; $T_e \leftarrow e \setminus H_e$;
Recompute $\vec{d}_v$ from the updated head sets;
$S \leftarrow \{v \in V : \vec{d}_v \geq k\}$;
$D_{k,\delta} \leftarrow$ vertices reached by reverse traversal from S in the directed incidence graph;
return $D_{k,\delta}$;

contains the two augmenting paths $s \to u_4 \to z_{e_2} \to u_1 \to t, s \to u_5 \to z_{e_4} \to u_2 \to t$. They carry two units of flow in total, which is maximum because the total capacity into t is two. The first path replaces u_1 by u_4 in the head set of e_2, while the second replaces u_2 by u_5 in the head set of e_4. The resulting indegrees are $(1,1,0,1,1)$, as illustrated in Figure 3(c). No vertex now has indegree at least two, so $S_2=\emptyset$ and $D_{2,1}=\emptyset$.

Algorithm 2 first constructs a valid δ-orientation and initializes the flow network at pivot $q=k-1$. The hyper-

edge arcs encode feasible tail-to-head exchanges, while the source and sink arcs specify how many contributions each vertex can receive or release. After computing an integral maximum flow, the algorithm identifies, for each e, the tail incidences U_e and head incidences W_e carrying one unit of flow. Flow conservation at z_e ensures $|U_e| = |W_e|$; replacing W_e with U_e therefore preserves $|H_e| = r_e$. This simultaneous update is equivalent to pairing the used tail and head incidences of each hyperedge, without reversing all incidences of the hyperedge. Finally, a reverse traversal from the vertices with indegree at least k returns all vertices that can reach such a seed.

Theorem 8 (Correctness of DSM-FLOW) *Algorithm 2 returns $D_{k,\delta}$.*

Proof. The flow is integral, and conservation at every factor vertex z_e gives $|U_e| = |W_e|$. The updated head set therefore contains r_e vertices, so the recovered orientation is valid. For each original vertex v, its new indegree is $\vec{d}'_v = \vec{d}_v + f(s,v) - f(v,t)$, where absent terminal arcs carry zero flow. Thus a vertex below q can only gain contributions, and a vertex above q can only lose them, without crossing q. After applying the flow, the residual incidence arcs represent precisely the tail-to-head exchanges allowed by the updated orientation. If a vertex of final indegree at most $k-2$ could reach another vertex of final indegree at least k, the residual source and sink capacities would extend that directed path to an augmenting s–t path. This contradicts the maximality of f. Hence the final orientation satisfies Lemma 5, and the reverse traversal in Algorithm 2 returns $D_{k,\delta}$. □

Theorem 9 (Conservative Flow Complexity) *With a generic Dinic max-flow algorithm, Algorithm 2 uses $O((n+m)^2(n+L))$ time and $O(n+m+L)$ space.*

Proof. The network has $O(n+m)$ vertices and $O(n+L)$ arcs. The generic $O(|V_N|^2|A|)$ bound for Dinic dominates network construction, flow recovery, and the final reverse traversal, which all take $O(n+m+L)$ time. The network and auxiliary arrays require $O(n+m+L)$ space. □

3.3 Greedy Initialization: DSM-FLOW+

DSM-FLOW constructs an arbitrary δ-orientation, which may concentrate hyperedge contributions on vertices with high indegrees and increase the subsequent flow computation. DSM-FLOW+ improves this initialization through a greedy head assignment strategy. Specifically, hyperedges are processed sequentially. For each hyperedge e, DSM-FLOW+ selects $r_e = \min\{\delta, |e|\}$ incident vertices with the smallest current indegrees as its head set H_e, breaking ties by vertex ID. The indegrees of the selected vertices are updated immediately before processing the next hyperedge. This strategy considers previously assigned contributions and reduces the initial imbalance among vertices. After constructing the δ-orientation, DSM-FLOW+ follows the same maximum-flow and reachability procedures as DSM-FLOW to compute $D_{k,\delta}$.

Algorithm 3: DSM-FLOW+$(\mathscr{H}, k, \delta)$

```
Data: H = (V,E), integers k ≥ 0, δ ≥ 1
Result: D_{k,δ}
if k = 0 then
    return V;
Initialize every indegree to zero;
foreach hyperedge e ∈ E do
    Select r_e = min{δ, |e|} incident vertices of lowest current
      indegree;
    Assign these r_e vertices to H_e, update their indegrees;
Run the flow and reachability steps of Algorithm 2;
return D_{k,δ};
```

Theorem 10 (Complexity of DSM-FLOW+) *Algorithm 3 is correct and has the same conservative asymptotic bounds as Algorithm 2, plus $O(\sum_e |e| \log |e|)$ time for sorting and choosing a prefix; a linear-time alternative is possible.*

Proof. Greedy initialization creates a valid δ-orientation. Algorithm 2 is correct from any valid initialization. □

4 Density Decomposition Algorithms

The algorithms in Section 3 compute $D_{k,\delta}$ for a given density level k. Applying them independently to every level would repeat substantial computation. Instead, we compute the complete IDN array from an egalitarian δ-orientation, where each dense set can be recovered as $D_{k,\delta} = \{v \in V : \bar{r}_v^{\delta} \geq k\}$. This avoids repeated searches and the explicit storage of nested dense sets.

We develop two algorithms, Flow-Decomp and Reverse-Decomp, both using greedy initialization, integral flow balancing, and residual-network reuse. Flow-Decomp computes maximum reachable indegrees through SCC condensation, while Reverse-Decomp uses descending-load reverse propagation. Once the orientation is balanced, both recover all IDNs in $O(n+m+L)$ time. For different values of δ, the decomposition can be computed independently over $\delta = 1, \ldots, d^E_{\max}$.

4.1 Integral Flow Balancing

We construct an initial δ-orientation using the greedy strategy of DSM-FLOW+. Starting with zero indegrees, we process each hyperedge e by selecting the $r_e = \min\{\delta, |e|\}$ incident vertices with the smallest current indegrees as its head

set, breaking ties by vertex ID. Their indegrees are updated before processing the next hyperedge. This procedure takes $O(\sum_e |e| \log(|e|+1))$ time using sorting and produces a valid δ-orientation, although further balancing may be required to obtain an egalitarian orientation.

For the current orientation, let $M(v) = \max_{t:v\rightsquigarrow t} \vec{d}_t$, including $t = v$. If some vertex s satisfies $M(s) \geq \vec{d}_s + 2$, a reversible hyperpath exists. We select an integer pivot q strictly between $\vec{d}_s$ and $M(s)$ and apply the reorientation flow network in Definition 12. The source and sink capacities are $\max\{0, q - \vec{d}_v\}$ and $\max\{0, \vec{d}_v - q\}$, respectively. Hyperedges with $|e| \leq \delta$ have no tails and require no movable factor nodes, while their head contributions remain in the vertex indegrees. The network contains $O(n+m)$ vertices and $O(n+L)$ arcs.

Each unit of integral flow changes the head assignments along a directed hyperpath, increasing the indegree of a lower-load vertex and decreasing that of a higher-load vertex. Flow conservation preserves the head budget of every hyperedge. Consequently, each unit of flow decreases the potential $\Psi = \sum_{v\in V} \vec{d}_v^2$ by at least two. The process terminates when $M(v) \leq \vec{d}_v + 1$ for all vertices, yielding an egalitarian δ-orientation. To avoid rebuilding the network in every iteration, we retain the residual capacities of the incidence arcs, which represent the updated orientation. Only the source and sink arcs are reset for the next pivot. The network is constructed when a reversible hyperpath is first detected and reused in subsequent balancing rounds.

Theorem 11 (Residual Network Reuse) *After each integral pivot flow, the incidence residual network represents a valid δ-orientation. Reusing this network in subsequent balancing rounds is equivalent to rebuilding it from the updated orientation and preserves the computed IDNs.*

Proof. Each incidence is represented by two opposite residual arcs with complementary binary capacities. An integral flow updates these capacities without changing their sum. By flow conservation at each factor vertex z_e, the number of assigned heads remains r_e. Thus, the updated residual network represents a valid δ-orientation. In subsequent balancing rounds, the incidence residual arcs are retained, while the source and sink arcs are reset according to the new pivot and current indegrees. Since the retained arcs encode the updated head assignments, the resulting network is equivalent to one constructed from the current orientation. This equivalence holds throughout the balancing process. Upon reaching an egalitarian orientation, Theorem 4 ensures that both algorithms recover the correct IDNs. □

4.2 Flow-Decomp: SCC-Based Decomposition

Flow-Decomp computes the maximum reachable indegree of each vertex using strongly connected component (SCC) condensation. For the current δ-orientation, we first construct a directed incidence graph by introducing a factor vertex z_e for each hyperedge $e \in E$. For every $u \in T_e$ and $v \in H_e$, we add directed arcs $u \to z_e$ and $z_e \to v$, respectively. These arcs preserve reachability in the directed hypergraph.

Algorithm 4: Flow-Decomp($\mathscr{H}, \delta$)

Data: Normalized hypergraph $\mathscr{H}$, integer $\delta \geq 1$
Result: Complete IDN array $\bar{r}^{\delta}$

```
Construct a greedy valid δ-orientation;
while true do
    Build the directed incidence graph and its SCC condensation;
    Compute reachable maxima M by reverse-topological propagation;
    if M(v) ≤ d_v + 1 for every v then
        return M;
    Select s with the largest reachable load gap M(s) − d_s ≥ 2;
    Set k ← max{d_s + 2, d_s + ⌈(M(s) − d_s)/2⌉} and q ← k − 1;
    Apply an integral maximum flow at pivot q, retaining incidence residual pairs;
```

We then compute the SCCs of the incidence graph and contract each SCC into a single vertex, forming a directed acyclic graph (DAG). For each component C, let $h(C)$ be the maximum indegree of its original vertices, or -1 if C contains only factor vertices. Since vertices within the same SCC have identical reachable components, their maximum reachable indegrees can be computed together. Specifically, we process the DAG in reverse topological order and compute $M(C) = \max\{h(C), \max_{C'\in N^+(C)} M(C')\}$, where $N^+(C)$ denotes the outgoing neighbors of C in the DAG. Each original vertex v belonging to C is then assigned $M(v) = M(C)$. If some vertex v satisfies $M(v) \geq \vec{d}_v + 2$, a reversible hyperpath exists, and Flow-Decomp selects a vertex with the largest indegree gap to perform flow balancing. The SCCs and reachable maxima are recomputed after each balancing round. When $M(v) \leq \vec{d}_v + 1$ holds for all vertices, no reversible hyperpath remains and the orientation is egalitarian. By Theorem 4, the computed values $M(v)$ are the IDNs. Since the incidence graph contains $O(n+m)$ vertices and $O(L)$ arcs, SCC computation and DAG propagation take $O(n+m+L)$ time per round.

Theorem 12 (Correctness of Flow-Decomp) *Algorithm 4 returns the canonical IDNs at δ. Its reachability computation costs $O(n+m+L)$ per round.*

Proof. Paths in the factor graph correspond to directed hyperpaths. Vertices in an SCC have the same reachable components, and DAG propagation computes their largest reachable indegree. Every positive flow preserves legality and strictly decreases Ψ. The stopping condition excludes

Algorithm 5: Reverse-Decomp($\mathscr{H}, \delta$)

Data: Normalized hypergraph $\mathscr{H}$, integer $\delta \geq 1$
Result: Complete IDN array $\bar{r}^{\delta}$

Construct a greedy valid δ-orientation;
while *true* **do**
  Build a head-to-edge CSR index for movable edges;
  Set $M(v) \leftarrow -1$; mark all factors unvisited;
  foreach *terminal t in nonincreasing indegree order* **do**
    if $M(t) = -1$ **then**
      Reverse-traverse from t, visiting each factor and vertex at most once; label reached vertices with $\vec{d}_t$;
  if $M(v) \leq \vec{d}_v + 1$ *for every* v **then**
    return M;
  Select s with the largest reachable load gap; choose k and $q = k - 1$ as in Algorithm 4;
  Apply an integral pivot flow; synchronize changed head incidences and retain residual pairs;

every reversible hyperpath. Thus the final orientation is egalitarian, and M equals the IDN array by Theorem 4. SCCs and the condensation have total size $O(n+m+L)$. □

4.3 Reverse-Decomp: Direct Reverse Propagation

For each movable edge, a head-to-edge CSR index supports reverse traversal from its heads to its tails. Process potential terminals in nonincreasing indegree order. Starting from an unlabeled terminal t, visit its unlabeled reverse predecessors and assign them $\vec{d}_t$. Mark each vertex and each factor only at its first visit. Edges with no tails can be omitted from this traversal while retaining their indegree contributions.

Theorem 13 (Correctness of Reverse-Decomp**)** *For every valid intermediate orientation, the reverse pass in Algorithm 5 computes $M(v) = \max_{t: v \rightsquigarrow t} \vec{d}_t$ in $O(n+m+L)$ time and space. The complete algorithm returns the canonical IDNs.*

Proof. Reverse traversal from t visits precisely vertices that can reach t. When a vertex is first labeled from t, any reachable terminal of larger indegree has already been processed and would have labeled that vertex earlier. Thus its first label is its maximum reachable indegree. Reusing a visited factor cannot improve a label because later terminals have no larger load. Every vertex and factor is visited once. Since $0 \leq \vec{d}_v \leq m$, counting buckets order terminals in $O(n+m)$ time; scanning the CSR and tails costs $O(L)$. The resulting witness and stopping condition are identical to those of Flow-Decomp, so the shared balancing argument and Theorem 4 prove correctness. □

4.4 Complexity Analysis

Theorem 14 (Complexity of Density Decomposition) *Let $B = n+m+L$, where $L = \sum_{e \in E} |e|$. For either Flow-Decomp or Reverse-Decomp, let q denote the number of positive flow-balancing rounds, and let $T^{(i)}_{\text{flow}}$ be the maximum-flow computation time in round i. Both algorithms require*

$$O\left(\sum_{e \in E} |e| \log(|e|+1) + (q+1)B + \sum_{i=1}^{q} T^{(i)}_{\text{flow}}\right)$$

time and $O(B)$ space. Moreover, $q \leq R_{\delta}^2/2$, where $R_{\delta} = \sum_{e \in E} \min\{\delta, |e|\}$. Using Dinic's algorithm, $T^{(i)}_{\text{flow}} = O((n+m)^2(n+L))$.

Proof. Both algorithms begin with greedy initialization, which processes each hyperedge by sorting its incident vertices according to their current indegrees. This requires $O(\sum_{e \in E} |e| \log(|e|+1))$ time. In each balancing round, Flow-Decomp computes the maximum reachable indegrees through SCC condensation and DAG propagation, while Reverse-Decomp uses direct reverse traversal. Both procedures take $O(B)$ time. The same bound applies to the final IDN computation. Hence, the total cost of reachability computation over q balancing rounds and the final extraction is $O((q+1)B)$. Each balancing round constructs or updates a flow network with $O(n+m)$ vertices and $O(n+L)$ arcs. Using Dinic's algorithm, the maximum-flow computation takes $O((n+m)^2(n+L))$ time per round. To bound the number of rounds, consider the potential $\Psi = \sum_{v \in V} \vec{d}_v^2$. Every positive balancing round decreases Ψ by at least two while preserving the total indegree R_{δ}. Since $0 \leq \Psi \leq R_{\delta}^2$, the number of positive balancing rounds satisfies $q \leq R_{\delta}^2/2$. Finally, both algorithms store the hypergraph incidences, vertex indegrees, auxiliary reachability structures, and the residual flow network, requiring $O(B)$ space. Combining these bounds establishes the theorem. □

5 Sandwich Bounds and Acceleration

The algorithms in Section 4 compute the complete density decomposition through global flow balancing. However, some vertices can be assigned to their density levels without completing the balancing process. Based on this observation, we introduce an optional optimization for both Flow-Decomp and Reverse-Decomp. We derive lower and upper reachability bounds that identify vertices whose membership can be determined in advance and restrict the remaining computation to a smaller set of vertices. We further show that the resulting subproblem preserves the original capped hyperedge contribution objective, allowing either decomposition algorithm to operate on the reduced instance without affecting correctness.

5.1 Lower and Upper Reachability Bounds

Consider any legal δ-orientation, not necessarily egalitarian, and include the length-zero path in reachability. Define

$$\ell(v) = \min_{u:u\rightsquigarrow v} \vec{d}_u, \qquad u(v) = \max_{w:v\rightsquigarrow w} \vec{d}_w.$$

For an integer k, let $A_k = \{v : \ell(v) \geq k\}$ and $B_k = \{v : u(v) \geq k\}$. Both sets are predecessor-closed. The lower quantity is obtained by propagating minimum loads forward, while the upper quantity is the reachable maximum already used by the decomposition algorithms.

Lemma 8 (Contribution–Indegree Equality) *If C is predecessor-closed in a valid δ-orientation, then $F_\delta(C) = \sum_{v\in C} \vec{d}_v$.*

Proof. Consider any hyperedge $e \in E$ with head set H_e and tail set T_e. If $H_e \cap C \neq \emptyset$, the predecessor-closed property of C implies $T_e \subseteq C$. Thus, every vertex in $e \setminus C$ belongs to H_e, and the number of heads in C is $|H_e \cap C| = r_e - |e \setminus C| = \max\{0, r_e - |e \setminus C|\}$. Otherwise, if $H_e \cap C = \emptyset$, all r_e heads lie outside C, giving $|e \setminus C| \geq r_e$. Hence, both the number of heads in C and the capped contribution of e to C are zero. Therefore, the contribution of each hyperedge to $F_\delta(C)$ equals its number of heads in C. Summing over all hyperedge occurrences yields $F_\delta(C) = \sum_{e\in E} |H_e \cap C| = \sum_{v\in C} \vec{d}_v$, which proves the result. □

Theorem 15 (Sandwich theorem) *For every legal δ-orientation and integer $k \geq 0$, $A_k \subseteq D_{k,\delta} \subseteq B_k$. The inclusions may be strict and hold before global balancing.*

Proof. Write $Q = Q_{k,\delta}$. Every vertex of A_k has load at least k. For nonempty $X \subseteq A_k$, Lemma 8 and the fact that $F_\delta(A_k \setminus X)$ is at most the number of heads assigned to $A_k \setminus X$ give $F_\delta(A_k) - F_\delta(A_k \setminus X) \geq \sum_{v\in X} \vec{d}_v \geq k|X| > (k-1)|X|$. Thus $Q(A_k) > Q(A_k \cap Z)$ whenever $A_k \setminus Z$ is nonempty. If Z is a maximizer, supermodularity and $Q(A_k \cup Z) \leq Q(Z)$ imply $Q(A_k) \leq Q(A_k \cap Z)$, a contradiction. Hence A_k is contained in every maximizer and therefore in their intersection $D_{k,\delta}$. For $Y \subseteq V \setminus B_k$, every $v \in Y$ has $\vec{d}_v \leq k-1$. Lemma 8 and the allocation upper bound yield $F_\delta(B_k \cup Y) - F_\delta(B_k) \leq \sum_{v\in Y} \vec{d}_v \leq (k-1)|Y|$. For any maximizer Z, supermodularity together with this inequality shows that $Z \cap B_k$ is also a maximizer. The unique smallest maximizer is contained in it, so $D_{k,\delta} \subseteq B_k$. □

At an egalitarian orientation, $u(v)$ is the IDN by Theorem 4, and hence $B_k = D_{k,\delta}$ for every k. The lower certificate can remain strict because a predecessor path to a seed may contain a vertex of load $k-1$, so in general only $A_k \subseteq D_{k,\delta} = B_k$ is guaranteed. The upper bounds alone therefore finalize all memberships at an egalitarian orientation. Before that point, $B_k \setminus A_k$ contains every vertex whose membership is not certified by the two bounds.

5.2 Contribution-Preserving Reduction

Simply inducing the hypergraph on $B_k \setminus A_k$ is incorrect because F_δ counts partially contained edges. For any predecessor-closed $A \subseteq B$, put $R = B \setminus A$. Each original occurrence e becomes $e_R = e \cap R$ with residual budget

$$r_e^{A,B} = \min\{|e_R|, \max\{0, r_e - |e \setminus B|\}\}.$$

Occurrences with zero residual budget are omitted; equal residual occurrences remain distinct.

Theorem 16 (Contribution Equivalence) *Let $A \subseteq B$ be predecessor-closed vertex sets and $R = B \setminus A$. Let $F_\delta^{A,B}$ denote the capped hyperedge contribution function of the reduced hypergraph defined above. For any $Y \subseteq R$, we have $F_\delta(A \cup Y) - F_\delta(A) = F_\delta^{A,B}(Y)$. Moreover, restricting the current δ-orientation to R preserves the required head count $r_e^{A,B}$ for each retained hyperedge, yielding a valid orientation of the reduced hypergraph.*

Proof. We first prove the contribution identity. Consider any hyperedge $e \in E$, and let $c = |e \setminus B|$ and $s = |e_R|$, where $e_R = e \cap R$. Since $A \subseteq B$ and $Y \subseteq R = B \setminus A$, we have $|e \setminus (A \cup Y)| = c + |e_R \setminus Y|$, and $|e \setminus A| = c + s$. Thus, the difference in the contribution of e is $g_e(A \cup Y) - g_e(A) = \max\{0, r_e - c - |e_R \setminus Y|\} - \max\{0, r_e - c - s\}$, where $g_e(X) = \max\{0, r_e - |e \setminus X|\}$. We consider three cases according to the value of $r_e - c$. If $r_e - c \leq 0$, both terms are zero, and $r_e^{A,B} = 0$. Hence, the contribution difference is zero. If $0 < r_e - c < s$, the second term vanishes and $r_e^{A,B} = r_e - c$. The difference becomes $\max\{0, r_e^{A,B} - |e_R \setminus Y|\}$. If $r_e - c \geq s$, the two terms differ by $s - |e_R \setminus Y| = |e_R \cap Y|$. Since $r_e^{A,B} = s$, this is also equal to $\max\{0, r_e^{A,B} - |e_R \setminus Y|\}$. Therefore, in all cases, $g_e(A \cup Y) - g_e(A) = \max\{0, r_e^{A,B} - |e_R \setminus Y|\}$. Summing over all hyperedge occurrences proves $F_\delta(A \cup Y) - F_\delta(A) = F_\delta^{A,B}(Y)$.

We next prove that the restricted orientation remains valid. Since A and B are predecessor-closed, the edgewise equality established in Lemma 8 gives $|H_e \cap A| = g_e(A), |H_e \cap B| = g_e(B)$. As $R = B \setminus A$, the number of heads retained in R is $|H_e \cap R| = |H_e \cap B| - |H_e \cap A| = \max\{0, r_e - c\} - \max\{0, r_e - c - s\} = r_e^{A,B}$. Thus, each retained hyperedge has $r_e^{A,B}$ heads after restriction, with all remaining vertices forming its tail set. The resulting orientation satisfies the required head budgets and is therefore valid for the reduced hypergraph. □

Taking $A = A_k$ and $B = B_k$, Theorem 15 restricts the maximization to sets $A_k \cup Y$ with $Y \subseteq B_k \setminus A_k$. The residual identity shows that the smallest maximizing Y is the solution at k on the residual subproblem. Thus the full answer is $D_{k,\delta} = A_k \cup Y$.

Algorithm 6: Sandwich($\mathscr{H}, \delta, [a,b]$)

Data: An oriented hypergraph $\vec{\mathscr{H}}$, edge budgets r_e, and IDN interval $[a,b]$
Result: IDNs $\bar{r}_v^{\delta}$ for all $v \in V$

if $a = b$ **then**
  Assign $\bar{r}_v^{\delta} \leftarrow a$ for all $v \in V$;
  return;
Compute $\ell(v)$ and $u(v)$ by SCC condensation or reverse propagation;
if $u(v) \le \vec{d}_v + 1$ *for all* $v \in V$ **then**
  Assign $\bar{r}_v^{\delta} \leftarrow u(v)$ for all $v \in V$;
  return;
$k \leftarrow a + \lfloor (b-a)/2 \rfloor + 1$;
$A \leftarrow \{v : \ell(v) \ge k\}$; $B \leftarrow \{v : u(v) \ge k\}$;
Construct the reduced hypergraph on $B \setminus A$ with budgets $r_e^{A,B}$;
Apply integral flow at pivot $k-1$ and recover the residual dense set $Y_k \subseteq B \setminus A$ by reverse reachability;
$D_{k,\delta} \leftarrow A \cup Y_k$; lift the residual head assignments to the current orientation;
Construct the two contribution-preserving child subproblems induced by $D_{k,\delta}$ and $V \setminus D_{k,\delta}$;
Recursively process them with $[k,b]$ and $[a,k-1]$, respectively;
return *the computed IDNs*;

5.3 Sandwich-Accelerated Decomposition

Algorithm 6 recursively computes the density decomposition over an integer IDN interval $[a,b]$. At each step, it selects a density level k and computes the lower and upper reachability bounds using either SCC condensation or reverse propagation. These bounds determine the membership of vertices outside the undecided set $B_k \setminus A_k$, allowing the remaining computation to be restricted to a smaller subproblem. The algorithm then applies integral flow to determine $D_{k,\delta}$, using the adjusted hyperedge budgets to preserve the original contribution function. The resulting dense set partitions the current vertices into two disjoint subproblems, corresponding to IDN intervals $[k,b]$ and $[a,k-1]$. Each subproblem is processed recursively with updated hyperedge budgets. If the current orientation is already egalitarian, the remaining IDNs are obtained directly from the upper reachability bounds without further recursion.

Theorem 17 (Correctness and complexity) *Algorithm 6, instantiated with either* Flow-Decomp *or* Reverse-Decomp*, returns the canonical IDN array at δ. At one recursion depth, its residual subproblems contain at most n vertices and at most L incidences in total. The depth is at most $\lceil \log_2(K+1) \rceil$, where $K = \max_v d_v$; bound construction therefore costs $O((n+m+L)\log(K+1))$ in total, in addition to the flow calls on the restricted subproblems. Working space is $O(n+m+L)$ when parent storage is released before processing its children.*

Proof. Theorem 15 determines membership outside the undecided set, and Theorem 16 makes the residual problem at k identical to the original objective on all remaining candidates. Induction over the recursion tree proves every split at k and hence every returned IDN. Each split partitions the current vertices. An original incidence belongs to at most one child, so the aggregate size per depth is linear. Midpoint splitting gives the depth bound, and a depth-first worklist permits the stated space bound. □

6 Extension to Dynamic Updates

For a hyperedge e, let $g_e(X) = \max\{0, r_e - |e \setminus X|\}$ denote its contribution to a vertex set X. Under hyperedge insertion or deletion, the capped contribution function is updated as

$$F_{\delta}^{+}(X) = F_{\delta}(X) + g_e(X), \qquad F_{\delta}^{-}(X) = F_{\delta}(X) - g_e(X).$$

When deleting a hyperedge, only the specified edge is removed, while other hyperedges with identical vertex sets are retained. Newly introduced vertices are treated as isolated vertices in the original hypergraph, and deleting hyperedges does not remove their incident vertices. The updated dense sets are defined as the unique inclusionwise smallest maximizers of their corresponding updated objectives.

Theorem 18 (IDN Change Bounds) *For the insertion of a hyperedge and $k \ge 1$, $D_{k,\delta} \subseteq D_{k,\delta}^{+} \subseteq D_{k-1,\delta}$. For deletion, $D_{k+1,\delta} \subseteq D_{k,\delta}^{-} \subseteq D_{k,\delta}$. Consequently, inserting a hyperedge increases each vertex's IDN by at most one, while deleting a hyperedge decreases it by at most one.*

Proof. We first establish a general property of supermodular functions. Let Q and $Q+h$ be supermodular, where h is nondecreasing, and let A and B be their respective smallest maximizers. Suppose $A \not\subseteq B$. Since $A \cap B$ is a proper subset of A, we have $Q(A) > Q(A \cap B)$. By supermodularity, $Q(A \cup B) - Q(B) \ge Q(A) - Q(A \cap B) > 0$. As h is nondecreasing, this implies $(Q+h)(A \cup B) > (Q+h)(B)$, contradicting the optimality of B. Hence, $A \subseteq B$. For hyperedge insertion, both $g_e(X)$ and $|X| - g_e(X)$ are nondecreasing, since adding a vertex increases g_e by either zero or one. Applying the above property to $Q_{k,\delta}$ and $Q_{k,\delta}^{+} = Q_{k,\delta} + g_e$ gives $D_{k,\delta} \subseteq D_{k,\delta}^{+}$. Moreover, $Q_{k,\delta}^{+}(X) + |X| - g_e(X) = Q_{k-1,\delta}(X)$, which yields $D_{k,\delta}^{+} \subseteq D_{k-1,\delta}$. All these objectives are supermodular by the supermodularity of their corresponding capped contribution functions. For deletion, applying the insertion result to the updated hypergraph gives $D_{k,\delta}^{-} \subseteq D_{k,\delta}$ and $D_{k+1,\delta} \subseteq D_{k,\delta}^{-}$. The stated IDN bounds follow directly from these inclusions and the nesting of dense sets. □

The one-unit bounds concern IDNs, not layer densities. Updated membership can change both sets in the layer-density numerator; old layer averages cannot simply

Algorithm 7: INSERT$(\vec{\mathscr{H}}, \bar{r}^{\delta}, e)$

Data: An egalitarian δ-orientation $\vec{\mathscr{H}}$, vertex IDNs $\bar{r}^{\delta}$, and a new hyperedge e
Result: Updated orientation and IDNs

Initialize newly added vertices with zero indegree and IDN;
Order $e = (u_1, u_2, \ldots, u_{|e|})$ such that $\vec{d}_{u_i} \le \vec{d}_{u_{i+1}}$;
$r_e \leftarrow \min\{\delta, |e|\}$, $H_e \leftarrow \{u_1, \ldots, u_{r_e}\}$;
Insert $\vec{e} = (e \setminus H_e, H_e)$ into $\vec{\mathscr{H}}$;
foreach $v \in H_e$ **do**
    $\vec{d}_v \leftarrow \vec{d}_v + 1$;
$C \leftarrow$ all connected components containing vertices of e;
while *a reversible hyperpath* $s \rightsquigarrow t$ *exists in* C **do**
    Choose q with $\vec{d}_s < q < \vec{d}_t$;
    Construct or reset the flow network at q;
    Compute an integral maximum flow and update the orientation and indegrees;
Compute $\bar{r}^{\delta}_v$ for all $v \in V$ by SCC condensation or reverse propagation;
return $(\vec{\mathscr{H}}, \bar{r}^{\delta})$;

Algorithm 8: DELETE$(\vec{\mathscr{H}}, \bar{r}^{\delta}, e)$

Data: An egalitarian δ-orientation $\vec{\mathscr{H}}$, vertex IDNs $\bar{r}^{\delta}$, and a hyperedge e to be deleted
Result: Updated orientation and IDNs

Let $\vec{e} = (T_e, H_e)$ and $A \leftarrow e$;
Remove $\vec{e}$ from $\vec{\mathscr{H}}$;
foreach $v \in H_e$ **do**
    $\vec{d}_v \leftarrow \vec{d}_v - 1$;
$C \leftarrow$ all connected components containing vertices of A in the updated hypergraph;
while *a reversible hyperpath* $s \rightsquigarrow t$ *exists in* C **do**
    Choose q with $\vec{d}_s < q < \vec{d}_t$;
    Construct or reset the flow network at q;
    Compute an integral maximum flow and update the orientation and indegrees;
Compute $\bar{r}^{\delta}_v$ for all $v \in V$ by SCC condensation or reverse propagation;
return $(\vec{\mathscr{H}}, \bar{r}^{\delta})$;

be incremented or decremented at edited heads. Layer statistics, when requested, are reevaluated using the updated function and IDNs by Proposition 1.

Updating vertex indegrees alone is insufficient to maintain the IDNs. In particular, deleting a hyperedge may change the IDN of a vertex in its tail set even when its indegree remains unchanged. We therefore retain the current δ-orientation and eliminate reversible hyperpaths within the affected weakly connected components. For an updated hyperedge e, let $A = e$ denote its vertex set, recorded before deletion if necessary. After the update, let $C(A)$ be the union of weakly connected components in the updated undirected incidence graph that contain at least one vertex of A. If deleting an edge disconnects a component, each affected component still contains a vertex of the deleted edge. For batch updates, A includes the vertices of all modified hyperedges. Algorithms 7 and 8 retain the existing orientation and perform flow balancing only within $C(A)$. Once no reversible hyperpath remains, the updated IDNs are computed using SCC condensation or reverse propagation. Components outside $C(A)$ remain unchanged and require no further balancing.

Theorem 19 (Correct dynamic maintenance) *For any δ, Algorithms 7 and 8 terminate and recover an egalitarian orientation and correct IDNs. Repairs need only start inside $C(A)$. This is a correctness and locality-of-search guarantee,* not *a worst-case sublinear update-time guarantee.*

Proof. After each update, every hyperedge has the required number of heads, so the resulting orientation remains a valid δ-orientation. Since the updated contribution function is additive across weakly connected components, its smallest maximizer can be obtained from the smallest maximizers of individual components. Outside $C(A)$, neither the hypergraph nor its orientation changes. These components therefore remain egalitarian, and their IDNs are unchanged. Any new reversible hyperpath must lie within $C(A)$. Suppose such a path runs from s to t, with $a = \vec{d}_s$ and $b = \vec{d}_t$, where $b - a \ge 2$. We choose an integer k satisfying $a + 2 \le k \le b$ and set $q = k - 1$. The flow network at pivot q admits a positive flow that adjusts head assignments from vertices above q to those below q, without changing the head budget of any hyperedge. Each unit of flow decreases the potential $\Psi = \sum_{v \in V} \vec{d}_v^2$ by at least two. Since Ψ is a nonnegative integer, the balancing process terminates after finitely many steps. At termination, no reversible hyperpath remains in $C(A)$. As the other components are already egalitarian, the updated orientation is egalitarian over the entire hypergraph. By Theorem 4, the final reachability computation yields the correct IDNs, hence the canonical dense sets of the updated hypergraph. □

The residual network can be reused across balancing rounds for the same hypergraph, but must be rebuilt after an insertion or deletion, since the factor vertices, incidence structure, and assigned head contributions may change. For each update, the flow network is constructed only when needed and reused in subsequent rounds. The potential Ψ decreases during balancing, but may increase or decrease after the next hyperedge update.

For successive updates, we retain the current orientation across batches to avoid repeated initialization. A chosen positive δ remains valid when $d^E_{\max}$ changes. If the parameter range expands, newly introduced values of δ require separate orientations. Identical hyperedges are treated as distinct records and deleted using stable IDs, while repeated vertices within a hyperedge are removed during normalization. In the worst case, an update may affect the entire hypergraph.

Section 7.4 compares component-based maintenance with recomputation on the same updated graphs.

Theorem 20 (Complexity of Dynamic Maintenance) *Let M denote the total number of hyperedge slots allocated and L_a the number of active incidences in the updated hypergraph. For a normalized hyperedge insertion or deletion, let q be the number of flow-balancing rounds and $T_{\text{flow}}^{(i)}$ the maximum-flow computation time in round i. Algorithms 7 and 8 require*

$$O\left((q+1)(n+M+L_a)+\sum_{i=1}^{q} T_{\text{flow}}^{(i)}\right)$$

time and $O(n+M+L_a)$ space.

Proof. Both algorithms retain the current δ-orientation and update only the hyperedges affected by an insertion or deletion. Identifying the affected connected components requires $O(n+M+L_a)$ time in the worst case. Each balancing round computes reachable indegrees using SCC condensation or reverse propagation, taking $O(n+M+L_a)$ time. The corresponding maximum-flow computation takes $T_{\text{flow}}^{(i)}$ time. After q rounds, the final IDNs are computed in $O(n+M+L_a)$ time. Summing these costs gives the stated time bound. The maintained hypergraph, vertex indegrees, incidence structures, and flow network require $O(n+M+L_a)$ space. Deleted hyperedges retain their identifiers but release their incidence storage. Thus, the space bound depends on the number of allocated hyperedge slots and active incidences rather than all historical incidences. □

7 Experimental Evaluation

7.1 Experimental setup

We evaluate the proposed algorithms on multiple datasets. All programs are written in C++ and compiled with GCC 11.1.0 using `-O3`. Experiments are conducted on a Linux server with a 2.9 GHz AMD Ryzen 3990X CPU and 256 GB RAM running CentOS 7.9.2 (64-bit).

Datasets. We evaluate our methods on nine benchmark hypergraphs, whose normalized statistics are summarized in Table 1. The first seven datasets are available from the Cornell Hypergraph Repository at `https://www.cs.cornell.edu/~arb/data/`. HC (*house-committees*) represents memberships of US legislators in congressional committees. CP (*contact-primary-school*) and CH (*contact-high-school*) record groups of students observed in face-to-face proximity. SB (*senate-bills*) and HB (*house-bills*) group legislators who co-sponsor the same bill. TC (*trivago-clicks*) groups hotels clicked within the same browsing session, whereas SA (*stackoverflow-answers*) groups questions answered by the same user. The remaining two datasets use pinned XGI HIF releases [30,31]. TM (*threads-math-sx*) groups users participating in Math Stack Exchange threads, and DBLP groups the authors of each publication.

Table 1 Data hypergraphs.

Data	n	m	L	max $\lvert e\rvert$
HC	1,290	341	11,843	81
CP	242	12,704	30,729	5
CH	327	7,818	18,192	5
SB	294	29,157	232,147	99
HB	1,494	60,987	1,248,666	399
TC	172,738	233,202	726,861	85
SA	15,211,989	1,103,243	26,109,177	61,315
TM	176,445	719,792	1,610,393	21
DBLP	1,930,378	3,700,681	10,326,773	280

Table 2 Subhypergraph search time.

Data	DSM-PATH	DSM-FLOW	DSM-FLOW+
HC	2.073	0.005	0.005
CP	0.009	0.009	0.010
CH	0.006	0.006	0.007
SB	3776.991	0.040	0.042
HB	TO	0.147	0.159
TC	7952.924	0.741	0.660
SA	TO	25.972	25.414
TM	7997.339	0.597	0.594
DBLP	TO	6.581	6.500

Algorithms. We evaluate three (k,δ)-dense subhypergraph search algorithms: DSM-PATH, DSM-FLOW, and DSM-FLOW+ (Algorithms 1–3). The two complete decomposition algorithms are Flow-Decomp and Reverse-Decomp (Algorithms 4 and 5). The Sandwich procedure in Algorithm 6 supplies certified bounds to either decomposition algorithm; the resulting executions are denoted Flow-S and Reverse-S. Algorithms 7 and 8 maintain the decomposition after hyperedge insertions and deletions. We compare the resulting hierarchy with degree core [37], nbr-k-core [2], (k,h)-core [38], and (α,β)-core [15]. We use $h=5$, $\beta=5$, and $\delta=5$ in the cross-model experiments.

7.2 Efficiency Evaluation

Exp-1: Runtime of dense subhypergraph search algorithms. Table 2 compares DSM-PATH, DSM-FLOW, and DSM-FLOW+ (Algorithms 1–3) on nine hypergraphs with $k=5$ and $\delta=5$. Here, TO indicates that an algorithm fails to return a result within 12 hours. The results show that both flow-based algorithms substantially outperform DSM-PATH, particularly on large hypergraphs. On SB, DSM-PATH takes 3776.991 seconds, whereas DSM-FLOW completes in only 0.040 seconds. Similar improvements

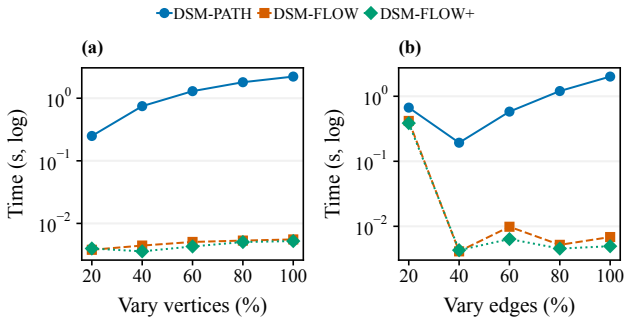


Fig. 4 Search scalability on HC.

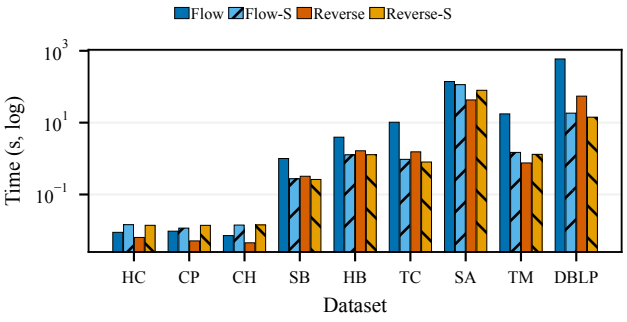


Fig. 5 Decomposition time on nine datasets.

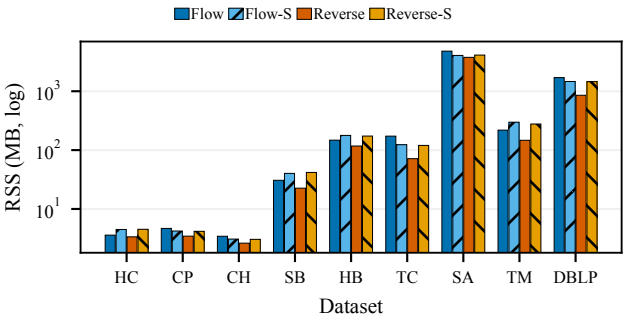


Fig. 6 Peak memory on nine datasets.

are observed on TC and TM, where DSM-FLOW finishes within one second, compared with over 7900 seconds for DSM-PATH. On HB, SA, and DBLP, DSM-PATH times out, while both flow-based algorithms complete successfully. These improvements stem from their different balancing strategies. DSM-PATH repeatedly searches for and reverses individual hyperpaths to obtain a globally egalitarian orientation, resulting in substantial repeated computation. In contrast, DSM-FLOW exploits the weaker balancing condition in Lemma 5 and uses integral maximum flow to adjust multiple head assignments simultaneously, avoiding repeated hyperpath searches and unnecessary global balancing. DSM-FLOW+ further improves the initial orientation through greedy head assignment, reducing the required flow adjustment in some cases. Figure 4 further examines the scalability of the three algorithms.

Exp-2: Scalability of subhypergraph mining algorithms. Figure 4 reports the scalability of DSM-PATH, DSM-FLOW, and DSM-FLOW+ on HC by varying the numbers of vertices and hyperedges. Similar trends are observed on the other datasets. Both flow-based algorithms consistently outperform DSM-PATH, with the performance gap becoming more pronounced as the hypergraph grows. At full scale, DSM-PATH takes 2.217 and 2.008 seconds in the vertex and hyperedge experiments, respectively, whereas both flow-based algorithms complete in less than 0.01 seconds. Their running times remain relatively stable as the hypergraph size increases, indicating better scalability. The performance difference mainly arises from their reorientation strategies.

Exp-3: Runtime of decomposition algorithms. Figure 5 compares Flow-Decomp and Reverse-Decomp, together with their Sandwich-accelerated variants (Flow-S and Reverse-S), on nine hypergraphs. Reverse-Decomp consistently outperforms Flow-Decomp on the five model-comparison datasets, achieving speedups of 3.11× on SB and 2.39× on HB. This advantage mainly comes from their different reachability computations. Flow-Decomp repeatedly constructs SCCs and propagates maximum indegrees over the condensation DAG, whereas Reverse-Decomp uses descending-load reverse propagation to compute the same results without SCC construction. This reduces the overhead of repeated reachability computations during flow balancing. Sandwich acceleration further improves efficiency by identifying vertices whose density levels can be determined in advance and restricting the remaining computation to equivalent reduced hypergraphs. On SB, Flow-S and Reverse-S take 0.274 and 0.262 seconds, respectively, while on HB they take 1.270 and 1.280 seconds. These results show that reachability-based reduction can reduce the cost of flow balancing. Both accelerated variants return the same IDNs as their original algorithms.

Exp-4: Memory overhead of decomposition algorithms. Figure 6 compares the peak memory consumption of the four decomposition methods on nine hypergraphs. Reverse-Decomp generally requires less memory than Flow-Decomp, particularly on larger datasets. On HB, its peak memory decreases from 147.56 MB to 117.34 MB, a reduction of 20.5%. This improvement mainly comes from avoiding explicit SCC construction and condensation DAG storage. Instead, Reverse-Decomp uses a compact reverse incidence index and directly propagates reachable indegrees, reducing the auxiliary data structures required during balancing. The Sandwich-accelerated variants maintain comparable memory consumption while reducing the computation through reachability bounds and smaller hypergraph instances. These results show that the proposed

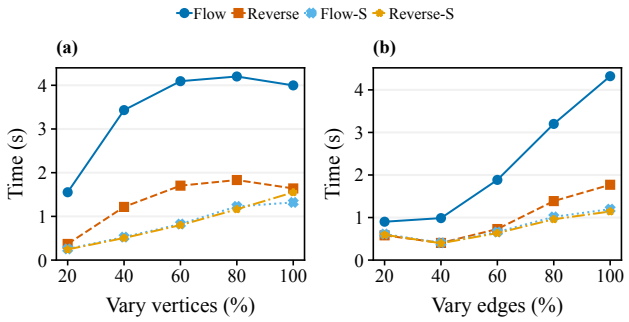


Fig. 7 Decomposition scalability on HB.

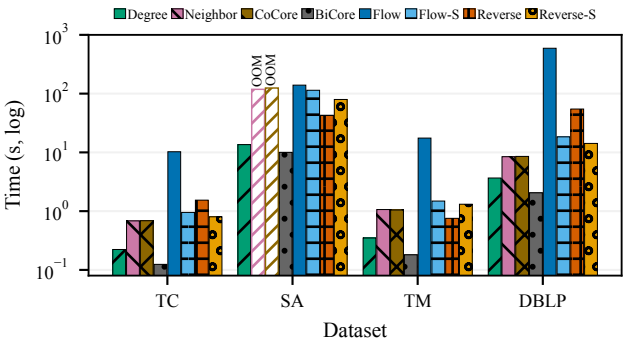


Fig. 8 Decomposition time on four large datasets.

optimizations improve computational efficiency without substantially increasing memory requirements.

Exp-5: Scalability with vertices and hyperedges. Figure 7 evaluates the scalability of Flow-Decomp, Reverse-Decomp, and their Sandwich-accelerated variants on HB by varying the numbers of vertices and hyperedges. Similar trends are observed on other datasets. As the hypergraph grows, Reverse-Decomp and the accelerated variants generally achieve lower running times than Flow-Decomp. At full scale, Flow-Decomp takes 3.998 and 4.322 seconds in the vertex and hyperedge experiments, respectively, compared with 1.639 and 1.770 seconds for Reverse-Decomp. Flow-S further reduces these times to 1.319 and 1.198 seconds, while Reverse-S takes 1.550 and 1.142 seconds. These improvements mainly result from direct reverse propagation, which avoids repeated SCC construction, and Sandwich acceleration, which reduces the vertices and hyperedges involved in flow balancing. Both techniques reduce computation as the hypergraph grows, improving the scalability of density decomposition.

Exp-6: Runtime comparison on large datasets. Figure 8 compares four established decomposition methods with our four algorithms on TC, SA, TM, and DBLP. The results show that our optimized algorithms can efficiently compute complete density decompositions on large hypergraphs. The fastest of our algorithms takes 0.801, 42.843, 0.755, and 14.209 seconds on these datasets, respectively. In particular, on SA, which contains over 15 million vertices, the complete decomposition finishes in 42.843 seconds. On DBLP, Reverse-S reduces the running time from 591.735 seconds for Flow-Decomp to 14.209 seconds, achieving a speedup of 41.6×. These improvements mainly come from direct reverse propagation and Sandwich acceleration. Reverse-Decomp avoids repeated SCC construction, while Sandwich acceleration identifies vertices whose density levels can be determined in advance and restricts flow balancing to smaller hypergraphs. Their combination substantially reduces the cost of complete decomposition on large datasets. Although the established methods have lower running times, they construct different core-based

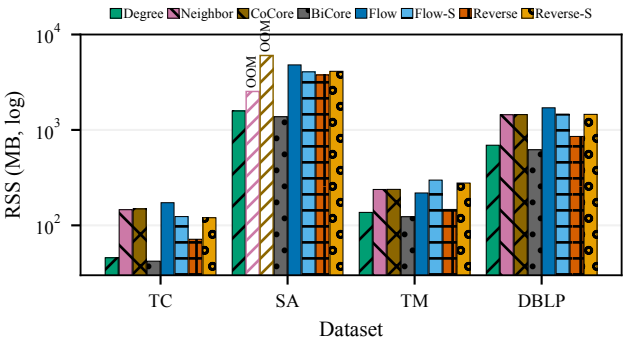


Fig. 9 Peak memory on four large datasets.

hierarchies rather than the proposed density decomposition based on capped hyperedge contributions. The results demonstrate that our optimizations make density-oriented hypergraph decomposition practical on large datasets.

Exp-7: Memory overhead on large datasets. Figure 9 compares the peak memory consumption of four established decomposition methods and our four algorithms on TC, SA, TM, and DBLP. Reverse-Decomp generally requires less memory than Flow-Decomp, particularly on TC and DBLP. This advantage comes from direct reverse propagation, which avoids explicit SCC construction and condensation DAG storage. The Sandwich-accelerated variants also maintain reasonable memory consumption on large hypergraphs. Moreover, all our algorithms successfully complete on SA, whereas Neighbor core and CoCore encounter memory exhaustion. These results demonstrate the memory efficiency and scalability of our decomposition algorithms.

7.3 Effectiveness Evaluation

Exp-8: Hierarchy structure across models. Figure 10 compares the hierarchical structures produced by different decomposition models in terms of total indexed levels, non-empty levels, and adjacent-layer consistency. Total indexed levels represent the range of density scores, while non-empty levels measure the number of distinct layers.

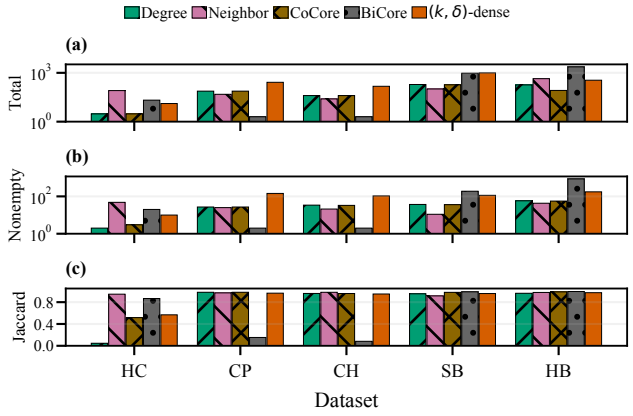


Fig. 10 Hierarchy statistics across decomposition models.

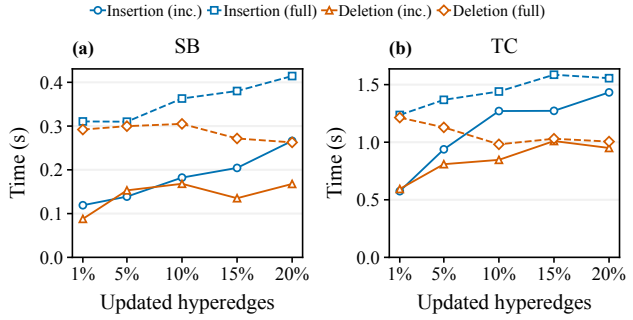


Fig. 11 Dynamic maintenance on SB and TC.

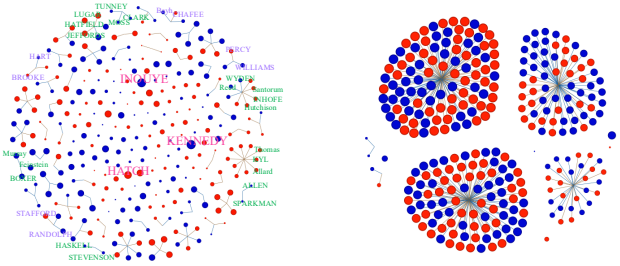
Adjacent-layer consistency is measured by the average Jaccard similarity between consecutive non-empty nested sets, reflecting their structural overlap. The proposed model produces more non-empty levels than degree core on all five datasets, indicating a finer distinction of density levels. This is because degree core relies on local incidence counts, whereas our model considers capped hyperedge contributions and balanced indegrees to distinguish density variations arising from higher-order interactions. Other models may produce more levels under different structural constraints. For example, neighborhood core has 48 levels on HC compared with our 10, while bipartite core has 862 levels on HB compared with our 175. These results illustrate the differences in hierarchical structure captured by the decomposition models.

7.4 Dynamic Updates

Exp-9: Dynamic maintenance. Figure 11 compares incremental maintenance with full recomputation under hyperedge insertions and deletions on SB and TC. The experiments consider batch updates involving 1%, 5%, 10%, 15%, and 20% of the original hyperedges. Incremental maintenance consistently outperforms full recomputation, achieving speedups ranging from 1.02× to 3.32× across all tested settings. The advantage comes from retaining the current egalitarian orientation and avoiding full initialization; component-local balancing can further reduce work when updates do not span the whole hypergraph. Incremental maintenance produces identical IDNs to full recomputation across all tested updates, confirming that our dynamic algorithms improve efficiency while maintaining the correctness of the density decomposition.

Table 3 Top-50 fraud screening and full-ranking metrics on AMLSim.

Model	TP	Precision	Recall	F1	ROC-AUC	AP
(k,δ)-Dense	28	0.560	0.0166	0.0323	0.9410	0.6411
Degree core	10	0.200	0.0059	0.0115	0.9391	0.6088
Neighbor core	13	0.260	0.0077	0.0150	0.9412	0.6182



(a) $(k,10)$-dense (b) nbr-k-core

Fig. 12 Co-sponsorship community visualization.

7.5 Case Studies

Exp-10: Fraud screening in multi-party transactions. We recover the AMLSim account labels, transaction stream, and alert groups used in the original case study [1]. Each alert is a hyperedge over its participating accounts. The proposed score is the current Reverse-Decomp IDN at $\delta = 5$; degree core and neighborhood core provide comparison scores. The released evaluator combines the score percentile with local transaction activity and applies the same Top-50 review budget to all three models. Each model is generated and evaluated once. Table 3 reports Top-50 classification metrics together with full-ranking ROC-AUC and average precision (AP). The proposed hierarchy places 28 fraudulent accounts in the first 50 positions, compared with 10 for degree core and 13 for neighborhood core. Its Top-50 precision is therefore 0.56, and its full-ranking AP is 0.6411. The three ROC-AUC values lie between 0.9391 and 0.9412. This case supports concentrated early retrieval and a higher precision–recall ranking under the released evaluator; it does not show a uniformly higher ROC-AUC.

Exp-11: Legislative co-sponsorship community discovery. We evaluate our method on the U.S. Senate Bill dataset, where each bill forms a hyperedge connecting its sponsors.

Legislators are represented as vertices and labeled by party affiliation (red for Republicans and blue for Democrats). This dataset captures real political collaborations and allows us to examine the community structures identified by different decomposition models. Figure 12 compares our $(k,10)$-dense decomposition ($\delta = 10 = \lfloor \bar{d}_e \rfloor$) with nbr-$k$-core. Our model identifies fine-grained communities that distinguish the two political parties while revealing cross-party collaborations. In particular, it highlights influential bipartisan legislators such as *Hatch*, *Kennedy*, and *Inouye*, as well as closely connected within-party groups involving *Murray*, *Feinstein*, and *Boxer*. In contrast, nbr-k-core merges these structures into coarser communities, making it difficult to distinguish different collaboration patterns. These results show that our density-based decomposition captures finer hierarchical structures and better reveals political collaboration patterns in legislative hypergraphs.

8 Related Work

Graph Decomposition. Graph decomposition has been extensively studied for identifying cohesive and dense structures. Representative cohesive models include k-core [4, 33,43], k-truss [12,52,23], nucleus decomposition [48, 49], and k-edge-connected components [9,8]. Extensions incorporating distance and attribute constraints include distance-generalized cores [6,14] and colorful h-star k-core decomposition [16,17]. Density-oriented approaches include densest subgraph discovery [18,10,3], locally densest subgraphs [46,50], higher-order dense subgraph models [51], density-friendly decomposition [56], and bipartite density decomposition [55]. Recent studies further investigate density search over relational graphs, integral density objectives on directed graphs, and locally dense h-clique structures [44,54,57]. Unlike these methods, which focus on pairwise relations or graph motifs, our work considers hyperedges directly and constructs a density hierarchy based on their capped contributions.

Cohesive and Density-Based Hypergraph Decomposition. Hypergraph decomposition extends cohesive structure analysis from pairwise relations to higher-order interactions. Representative core-based models include hypergraph k-core [47,5], nbr-k-core [2,53], (k,h)-core [38], (k,g)-core [27], (k,q)-core [39,32], and (k,t)-core [7]. These models characterize cohesiveness through degree, neighborhood, co-occurrence, hyperedge-size, and participation constraints. Other approaches, including densest subhypergraph discovery [22,24] and hypergraph truss decomposition [45], study density and higher-order structural cohesion. Recent studies have further explored constrained-core indexing, dynamic core maintenance, community search, locality-based indexing, decay-aware cores, refined (k,g)-core decomposition, and overlap-bounded decomposition [40,34,35,29,28,26,36]. Unlike core-based methods that primarily impose structural constraints, our model defines a nested density hierarchy through capped hyperedge contributions and balanced orientations, allowing density variations to be characterized directly.

Structural hypergraph decomposition. Structural hypergraph decompositions focus on acyclicity, constraint satisfaction, and query tractability. Representative frameworks include k-hinge tree decomposition [25], hypertree decomposition [20], generalized hypertree decomposition [11], fractional hypertree decomposition [21,19], and submodular-width based decomposition [42]. These methods target structural tractability, whereas our work focuses on density-aware hierarchical mining in hypergraphs.

9 Conclusion

In this paper, we studied density decomposition in hypergraphs and proposed the (k,δ)-dense subhypergraph model based on capped hyperedge contributions. We established the uniqueness and nesting properties of the resulting dense sets and derived an equivalent characterization using egalitarian δ-orientations and reachability. Based on these properties, we developed three dense subhypergraph search algorithms, DSM-PATH, DSM-FLOW, and DSM-FLOW+, and two decomposition algorithms, Flow-Decomp and Reverse-Decomp. We further introduced sandwich bounds to reduce the computation to equivalent smaller hypergraphs and developed incremental algorithms for maintaining density levels under hyperedge insertions and deletions. Extensive experiments on nine real-world hypergraphs demonstrate the efficiency and scalability of our algorithms. In particular, flow-based search substantially reduces the cost of hyperpath reorientation, while reverse propagation and sandwich acceleration improve decomposition efficiency. Incremental maintenance further reduces update time while producing the same results as full recomputation. Comparisons with established decomposition models and two case studies show that our model captures fine-grained density structures that conventional degree-based methods may overlook.

Statements and Declarations

Funding. This work was partially supported by the New Generation Artificial Intelligence National Science and Technology Major Project (2025ZD0123403), the National Natural Science Foundation of China (NSFC) under Grants 62572055 and 62372342, and the BIT Kunpeng&Ascend Center of Cultivation.

Competing interests. The authors have no relevant financial or non-financial interests to disclose.

References

1. Altman, E.R., Blanusa, J., von Niederhäusern, L., Egressy, B., Anghel, A., Atasu, K.: Realistic synthetic financial transactions for anti-money laundering models. In: NeurIPS (2023)
2. Arafat, N.A., Khan, A., Rai, A.K., Ghosh, B.: Neighborhood-based hypergraph core decomposition. VLDB **16**(9), 2061–2074 (2023)
3. Bahmani, B., Kumar, R., Vassilvitskii, S.: Densest subgraph in streaming and mapreduce. Proc. VLDB Endow. **5**(5), 454–465 (2012)
4. Batagelj, V., Zaversnik, M.: An o(m) algorithm for cores decomposition of networks. CoRR **cs.DS/0310049** (2003)
5. Bianconi, G., Dorogovtsev, S.N.: Nature of hypergraph k-core percolation problems. Physical Review E **109**(1), 014,307 (2024)
6. Bonchi, F., Khan, A., Severini, L.: Distance-generalized core decomposition. In: SIGMOD, pp. 1006–1023 (2019)
7. Bu, F., Lee, G., Shin, K.: Hypercore decomposition for non-fragile hyperedges: concepts, algorithms, observations, and applications. Data Min. Knowl. Discov. **37**(6), 2389–2437 (2023)
8. Chang, L., Wang, Z.: A near-optimal approach to edge connectivity-based hierarchical graph decomposition. VLDB J. **33**(1), 49–71 (2024)
9. Chang, L., Yu, J.X., Qin, L., Lin, X., Liu, C., Liang, W.: Efficiently computing k-edge connected components via graph decomposition. In: SIGMOD, pp. 205–216 (2013)
10. Charikar, M.: Greedy approximation algorithms for finding dense components in a graph. In: APPROX, pp. 84–95 (2000)
11. Cohen, D.A., Jeavons, P., Gyssens, M.: A unified theory of structural tractability for constraint satisfaction problems. J. Comput. Syst. Sci. **74**(5), 721–743 (2008)
12. Cohen, J.: Trusses: Cohesive subgraphs for social network analysis. National security agency technical report **16**(3.1), 1–29 (2008)
13. Contisciani, M., Battiston, F., Bacco, C.D.: Principled inference of hyperedges and overlapping communities in hypergraphs. CoRR **abs/2204.05646** (2022)
14. Dai, Q., Li, R., Qin, L., Wang, G., Yang, W., Zhang, Z., Yuan, Y.: Scaling up distance-generalized core decomposition. In: CIKM, pp. 312–321 (2021)
15. Ding, D., Li, H., Huang, Z., Mamoulis, N.: Efficient fault-tolerant group recommendation using alpha-beta-core. In: CIKM, p. 2047–2050 (2017)
16. Gao, S., Li, R., Qin, H., Chen, H., Yuan, Y., Wang, G.: Colorful h-star core decomposition. In: ICDE, pp. 2588–2601 (2022)
17. Gao, S., Qin, H., Li, R., He, B.: Parallel colorful h-star core maintenance in dynamic graphs. VLDB **16**(10), 2538–2550 (2023)
18. Goldberg, A.V.: Finding a maximum density subgraph. Tech. Rep. UCB/CSD-84-171, EECS Department, University of California, Berkeley (1984)
19. Gottlob, G., Lanzinger, M., Pichler, R., Razgon, I.: Complexity analysis of generalized and fractional hypertree decompositions. J. ACM **68**(5), 38:1–38:50 (2021)
20. Gottlob, G., Leone, N., Scarcello, F.: Hypertree decompositions and tractable queries. J. Comput. Syst. Sci. **64**(3), 579–627 (2002)
21. Grohe, M., Marx, D.: Constraint solving via fractional edge covers. ACM Trans. Algorithms **11**(1), 4:1–4:20 (2014)
22. Hu, S., Wu, X., Chan, T.H.: Maintaining densest subsets efficiently in evolving hypergraphs. In: CIKM, pp. 929–938 (2017)
23. Huang, X., Cheng, H., Qin, L., Tian, W., Yu, J.X.: Querying k-truss community in large and dynamic graphs. In: SIGMOD, pp. 1311–1322 (2014)
24. Huang, Y., Gleich, D.F., Veldt, N.: Densest subhypergraph: Negative supermodular functions and strongly localized methods. In: WWW, pp. 881–892 (2024)
25. Jeavons, P., Cohen, D., Gyssens, M.: A structural decomposition for hypergraphs. Contemporary Mathematics **178**, 161–177 (1994)
26. Kim, D., Kim, H., Kim, S., Kim, M., Kim, J., Lee, Y., Lim, S.: Uncovering high-order cohesive structures: Efficient (k,g)-core computation and decomposition for large hypergraphs. Knowl. Based Syst. **335**, 115,195 (2026)
27. Kim, D., Kim, J., Lim, S., Jeong, H.J.: Exploring cohesive subgraphs in hypergraphs: The (k, g)-core approach. In: CIKM, pp. 4013–4017 (2023)
28. Kim, H., Kim, M., Kim, D., Kim, J.: When user engagement meets structural cohesiveness: A decay-driven approach to hypergraph cores. In: CIKM, pp. 4879–4883 (2025)
29. Kim, S., Kim, D., Han, T., Kim, J., Jeong, H.J., Kim, J.: Efficient locality-based indexing for cohesive subgraphs discovery in hypergraphs. In: EDBT, pp. 170–182 (2026)
30. Landry, N.: coauth-dblp: HIF dataset, version 0.2. Zenodo (2026)
31. Landry, N.: threads-math-sx: HIF dataset, version 0.1. Zenodo (2026)
32. Lee, J., Goh, K.I., Lee, D.S., Kahng, B.: (k, q)-core decomposition of hypergraphs. Chaos, Solitons & Fractals **173**, 113,645 (2023)
33. Li, R., Yu, J.X., Mao, R.: Efficient core maintenance in large dynamic graphs. IEEE Trans. Knowl. Data Eng. **26**(10), 2453–2465 (2014)
34. Liu, Y., Luo, Q., Zheng, Y., Cai, Z., Yu, D.: Efficient traversal for core maintenance in large-scale dynamic hypergraphs. World Wide Web **28**(4), 45 (2025)
35. Liu, Y., Luo, Q., Zheng, Y., Zhang, W., Lin, X., Yu, D.: Effective and efficient community search over large-scale hypergraphs. In: EDBT, pp. 722–734 (2025)
36. Luo, Q., Yang, Z., Zhang, W., Zhou, A., Yu, D., Cheng, X., Lin, X., Guo, S.: Hypergraph decomposition with intersection bounds. VLDB J. **35**(4), 34 (2026)
37. Luo, Q., Yu, D., Cai, Z., Lin, X., Cheng, X.: Hypercore maintenance in dynamic hypergraphs. In: ICDE, pp. 2051–2056 (2021)
38. Luo, Q., Yu, D., Liu, Y., Zheng, Y., Cheng, X., Lin, X.: Finer-grained engagement in hypergraphs. In: ICDE, pp. 423–435 (2023)
39. Luo, Q., Zhang, W., Yang, Z., Wen, D., Wang, X., Yu, D., Lin, X.: Hierarchical structure construction on hypergraphs. In: CIKM, pp. 1597–1606 (2024)
40. Luo, Q., Zhang, W., Yang, Z., Yu, D., Lin, X., Wang, L.: Efficient indexing and searching of constrained core in hypergraphs. VLDB J. **34**(3), 34 (2025)
41. Mancastroppa, M., Iacopini, I., Petri, G., Barrat, A.: Hyper-cores promote localization and efficient seeding in higher-order processes. CoRR **abs/2301.04235** (2023)
42. Marx, D.: Tractable hypergraph properties for constraint satisfaction and conjunctive queries. J. ACM **60**(6), 42:1–42:51 (2013)
43. Montresor, A., Pellegrini, F.D., Miorandi, D.: Distributed k-core decomposition. IEEE Trans. Parallel Distributed Syst. **24**(2), 288–300 (2013)
44. Niu, Y., Li, Y., Jiang, J., Lakshmanan, L.V.S.: SANS: Efficient densest subgraph discovery over relational graphs without materialization. In: WWW, pp. 4250–4263 (2025)
45. Qin, H., Zeng, G., Li, R., Lin, L., Yuan, Y., Wang, G.: Truss decomposition in hypergraphs. Proc. VLDB Endow. **18**(7), 2185–2197 (2025)
46. Qin, L., Li, R., Chang, L., Zhang, C.: Locally densest subgraph discovery. In: SIGKDD, pp. 965–974 (2015)
47. Ramadan, E.Y., Tarafdar, A., Pothen, A.: A hypergraph model for the yeast protein complex network. In: IPDPS (2004)
48. Sariyüce, A.E., Seshadhri, C., Pinar, A., Çatalyürek, Ü.V.: Finding the hierarchy of dense subgraphs using nucleus decompositions. In: WWW, pp. 927–937 (2015)

49. Shi, J., Dhulipala, L., Shun, J.: Theoretically and practically efficient parallel nucleus decomposition. VLDB **15**(3), 583–596 (2021)
50. Tatti, N.: Density-friendly graph decomposition. ACM Trans. Knowl. Discov. Data **13**(5), 54:1–54:29 (2019)
51. Tsourakakis, C.: The k-clique densest subgraph problem. In: WWW, pp. 1122–1132 (2015)
52. Wang, J., Cheng, J.: Truss decomposition in massive networks. VLDB **5**(9), 812–823 (2012)
53. Zhang, W., Yang, Z., Wen, D., Li, W., Zhang, W., Lin, X.: Accelerating core decomposition in billion-scale hypergraphs. Proc. ACM Manag. Data **3**(1), 6:1–6:27 (2025)
54. Zhang, Y., Li, R., Lin, L., Zhang, Q., Qin, L., Wang, G.: Integral densest subgraph search on directed graphs. Proc. ACM Manag. Data **3**(3), 176:1–176:26 (2025)
55. Zhang, Y., Li, R., Zhang, Q., Qin, H., Qin, L., Wang, G.: Density decomposition of bipartite graphs. Proc. ACM Manag. Data **3**(1), 30:1–30:25 (2025)
56. Zhang, Y., Li, R., Zhang, Q., Qin, H., Wang, G.: Efficient algorithms for density decomposition on large static and dynamic graphs. Proc. VLDB Endow. **17**(11), 2933–2945 (2024)
57. Zhou, Y., Huang, T., Fang, Y.: Efficient locally h-clique densest subgraph discovery via divide-and-conquer. Proc. VLDB Endow. **19**(10), 2577–2590 (2026)